



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Sistemas de Información

Desarrollo de una herramienta para la detección de vulnerabilidades en protocolos de distribución de claves simétricas

Development of a tool for vulnerability detection in symmetric key distribution protocols

Realizado por

Carlos Corbacho Ordóñez

Tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, Julio, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Mención en Sistemas de Información

**Desarrollo de una herramienta para la detección de
vulnerabilidades en protocolos de distribución de
claves simétricas**

**Development of a tool for vulnerability detection in
symmetric key distribution protocols**

Realizado por

Carlos Corbacho Ordóñez

Tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JULIO, 2026

Fecha defensa: Septiembre, 2026

Resumen

El objetivo de este Trabajo de Fin de Grado consiste en el diseño e implementación de una aplicación de escritorio orientada al análisis y verificación de la seguridad en protocolos de distribución de claves simétricas. A diferencia de los simuladores de red tradicionales, esta aplicación opera a nivel matemático abstrayendo conceptos criptográficos, como pueden ser los nonces y las claves, con el objetivo de evaluar vulnerabilidades teóricas de diseño. Para ello, el sistema se basa en la notación estándar de protocolos (Alice y Bob).

Presenta un motor de compilación propio desarrollado en Java que ejerce como núcleo del sistema. Este está compuesto por un Analizador Léxico, basado en expresiones regulares para la validación estructural, y un Analizador Sintáctico de Descenso Recursivo, que convierte el texto introducido por el usuario en un Árbol Sintáctico Abstracto empleando el patrón de diseño Composite.

A nivel de arquitectura de software, la aplicación está fundamentada en el patrón Modelo-Vista-Controlador (MVC), garantizando así la separación entre lógica e interfaz gráfica construida en JavaFX. Además, mediante SQLite se integra un sistema de persistencia de datos local, gestionando sesiones de usuario y protegiendo sus credenciales a través de funciones de derivación de claves (BCrypt).

Palabras clave: análisis de protocolos, seguridad, criptografía simétrica, Analizador de descenso recursivo, MVC, JavaFX.

Abstract

The objective of this Bachelor's Thesis is the design and implementation of a desktop application aimed at the analysis and verification of security in symmetric key distribution protocols. Unlike traditional network simulators, this application operates at a mathematical level by abstracting cryptographic concepts, such as nonces and keys, with the goal of evaluating theoretical design vulnerabilities. To achieve this, the system relies on the standard protocol notation (Alice and Bob).

It features a custom compilation engine developed in Java that serves as the core of the system. This engine consists of a Lexical Analyzer, based on regular expressions for structural validation, and a Recursive Descent Parser, which converts the text entered by the user into an Abstract Syntax Tree using the Composite design pattern.

At the software architecture level, the application is based on the Model-View-Controller (MVC), thus ensuring a strict separation between the business logic and the graphical user interface built in JavaFX. Furthermore, a local data persistence system is integrated using SQLite, managing user sessions and protecting their credentials through key derivation functions (BCrypt).

Keywords: Protocol Analysis, Security, Symmetric Cryptography, Recursive Descent Parser, MVC, JavaFX.

Agradecimientos

La finalización de este Trabajo de Fin de Grado representa el punto final de una etapa académica que ha conllevado un gran esfuerzo y dedicación, algo que no habría sido posible sin el apoyo incondicional de ciertas personas clave.

En primer lugar, quiero dar las gracias a mi tutor, el Dr. Francisco José Jaime Rodríguez, por su paciencia, su rapidez a la hora de resolver cualquier duda, su esfuerzo para encontrar siempre un hueco donde reunirnos a pesar de sus múltiples obligaciones, y por haberme guiado con tanta cercanía a lo largo de todo este proyecto.

A mis padres y a mis tíos, que a lo largo de estos años me han brindado un apoyo inquebrantable, que me han ayudado con todo lo que han podido y más, y que muchas veces han vivido y sufrido este proceso incluso con mayor intensidad que yo.

Un recuerdo muy especial para mi abuela y mi tía, a quienes les hubiera encantado verme concluir esta etapa y que siempre creyeron en mí y me apoyaron más que nadie.

Finalmente, a mis compañeros de piso y a mis amigos, con quienes he compartido las penas, alegrías y vivencias de todos estos años universitarios; recuerdos y aprendizajes que siempre llevaré conmigo.

Índice General

Resumen	1
Abstract	3
Agradecimientos	5
Índice General	7
Índice de Figuras	11
Índice de Tablas	13
1 Introducción	17
1.1 Contexto	17
1.2 Motivación	18
1.3 Objetivos	18
1.4 Metodología	19
2 Estado del Arte y Marco Teórico	21
2.1 Notación Criptográfica Estándar	21
2.2 Conceptos Criptográficos	22
2.2.1 Criptografía Simétrica	22
2.2.2 Nonces (Number used ONCE)	23
2.2.3 Frescura de los Mensajes	23
2.3 El Modelo del Intruso de Dolev-Yao	23
2.4 Teoría de Compiladores	24
2.4.1 Análisis Léxico	24
2.4.2 Análisis Sintáctico y Árbol Sintáctico Abstracto (AST)	25
2.5 Herramientas Existentes	26
3 Análisis del Sistema	27
3.1 Requisitos Funcionales	27
3.1.1 Requisitos de registro y cuentas de usuario	27
3.1.2 Requisitos del dashboard	28
3.1.3 Requisitos de archivo de protocolo	29
3.1.4 Requisitos de análisis y notificación de vulnerabilidades	30

3.1.5	Requisitos de importación y exportación	31
3.1.6	Requisitos de generación de informes y soporte	31
3.1.7	Requisitos de base de datos	31
3.2	Requisitos No Funcionales	32
3.2.1	Rendimiento y robustez	32
3.2.2	Usabilidad	32
3.2.3	Seguridad y arquitectura	32
3.3	Casos de Uso	33
3.3.1	Identificación de Actores	33
3.3.2	Diagrama de Casos de Uso	34
3.3.3	Especificación Detallada de Casos de Uso	34
4	Diseño y Arquitectura	49
4.1	Arquitectura General	49
4.1.1	Jerarquía de Excepciones del Dominio	52
4.2	Diseño de la Lógica (Motor)	53
4.2.1	Patrón Estructural: Composite	53
4.3	Diseño de Datos	54
4.3.1	Diccionario de Datos	54
4.4	Diseño de Interfaz	56
4.4.1	Acceso al Sistema	57
4.4.2	Panel de Control (Dashboard)	57
4.4.3	Entorno de Especificación y Análisis	58
4.4.4	Gestión de Usuario y Soporte Integrado	59
4.4.5	Seguridad en Acciones Irreversibles	59
5	Implementación	61
5.1	Tecnologías Utilizadas	61
5.1.1	Lenguaje de Programación: Java	61
5.1.2	Interfaz Gráfica de Usuario: JavaFX	62
5.1.3	Sistema de Persistencia: SQLite	62
5.1.4	Librerías y Dependencias Auxiliares	63
5.1.5	Gestión de Credenciales SMTP y Modelado de Amenazas	63
5.1.6	Herramientas de Desarrollo y Gestión	64

5.2	Desarrollo del Analizador	65
5.2.1	Analizador Léxico y Validación Estructural	65
5.3	Desarrollo del Motor Criptográfico	66
5.3.1	Soporte de Operaciones Funcionales sobre Nonces	67
5.3.2	Centralización de Tipos de Vulnerabilidad	68
5.3.3	Evaluación de Frescura y Prevención de <i>Replay Attacks</i>	68
5.3.4	Mecanismos de Desafío-Respuesta	68
5.3.5	Detección de Riesgo de Reflexión	69
5.3.6	Evaluación de Claves, Suplantación y Autenticación Mutua	69
5.3.7	Resultado Final de la Interfaz	70
6	Pruebas y Verificación	71
6.1	Trazabilidad de Requisitos	71
6.2	Metodología de Pruebas	73
6.3	Casos de Verificación	74
6.3.1	Verificación Léxico-Sintáctica y Manejo de Errores de Entrada	74
6.3.2	Detección de Ataques Reales mediante el Motor Analítico	75
6.3.3	Seguridad Local y Persistencia de Credenciales	77
6.3.4	Verificación Algorítmica Continua mediante TDD	77
7	Conclusiones y Trabajo Futuro	79
7.1	Conclusiones	79
7.2	Trabajo Futuro	80
A	Diagramas de Secuencia Adicionales	83
B	Diagrama de clases de Implementación	87
C	Diagrama del AST	95
D	Registro del Trello	97
	Bibliografía	99

Índice de Figuras

3.1	Diagrama general de Casos de Uso del sistema.	34
3.2	Diagrama de secuencia del Caso de Uso 10: Ejecución del motor criptográfico y evaluación de vulnerabilidades.	43
4.1	Paquete de Lógica del diagrama de clases de implementación y arquitectura por capas.	51
4.2	Paquete de Modelo del diagrama de clases de implementación y arquitectura por capas.	52
4.3	Diagrama de clases del Analizador Sintáctico y la estructura del AST (Patrón Composite) simplificado (versión completa en el anexo).	54
4.4	Modelo Entidad-Relación de la base de datos local SQLite.	56
4.5	Interfaz de Inicio de Sesión.	57
4.6	Interfaz de Registro de Cuenta.	57
4.7	Vista principal del panel de control (<i>Dashboard</i>).	58
4.8	Entorno del editor de texto y volcado de resultados analíticos.	58
4.9	Interfaz de gestión de cuenta.	59
4.10	Guía estática de sintaxis y uso.	59
4.11	Confirmación de borrado de protocolo.	59
4.12	Confirmación de eliminación de cuenta.	59
5.1	Pantalla de inicio de sesión final implementada en JavaFX.	70
5.2	Panel de control (<i>Dashboard</i>) con historial de protocolos cargado.	70
5.3	Editor de protocolos con especificación cargada y numeración de líneas.	70
5.4	Panel de resultados mostrando veredicto positivo tras el análisis.	70
6.1	Captura de pantalla de la aplicación interceptando y notificando un error de sintaxis en la especificación del protocolo.	75
6.2	Visualización en la interfaz del sistema tras detectar un fallo por falta de garantías de frescura en el intercambio de claves.	76

6.3	Diagnóstico interactivo de la GUI detallando la mitigación y alerta del ataque de suplantación de identidad por reflexión.	77
6.4	Interfaz de la aplicación mostrando el veredicto <i>Seguro</i> para el protocolo Yahalom tras superar todos los filtros del motor analítico.	78
A.1	Diagrama de secuencia correspondiente al proceso de Registro de Usuario CU01. . .	83
A.2	Diagrama de secuencia correspondiente al proceso de Inicio de Sesión CU02.	84
A.3	Diagrama de secuencia correspondiente al proceso de Eliminación de Cuenta CU06. .	84
A.4	Diagrama de secuencia correspondiente al proceso de Importación de un Protocolo CU12.	85
A.5	Diagrama de secuencia correspondiente al proceso de Generación de Informes en PDF CU14.	85
B.1	Diagrama de clases de implementación y arquitectura por capas.	88
B.2	Diagrama de clases de implementación y arquitectura por capas paquete modelo. . .	89
B.3	Diagrama de clases de implementación y arquitectura por capas paquete lógica. . . .	90
B.4	Diagrama de clases de implementación y arquitectura por capas paquete útil.	91
B.5	Diagrama de clases de implementación y arquitectura por capas paquete persistencia. .	92
B.6	Diagrama de clases de implementación y arquitectura por capas paquete presentación. .	93
C.1	Diagrama de clases del Analizador Sintáctico y la estructura del AST (Patrón Composite).	96
D.1	Registro de los sprints llevados a cabo durante la elaboración del proyecto.	97

Índice de Tablas

4.1	Esquema de la tabla USUARIO.	55
4.2	Esquema de la tabla PROTOCOLO.	55
4.3	Esquema de la tabla VULNERABILIDAD.	56
6.1	Tabla de trazabilidad: Requisito → Caso de Uso → Prueba de verificación.	72
6.2	Inventario completo de la suite de pruebas unitarias y resultado de ejecución con <i>Apache Maven Surefire</i>	74

Nomenclatura

$\{M\}_K$ Mensaje M cifrado mediante la clave K

A, B Agentes legítimos que participan en el protocolo (Alice y Bob)

$F(N)$ Función criptográfica aplicada sobre un nonce como mecanismo de desafío-respuesta

I Atacante o intruso (Intruder) en el modelo Dolev-Yao

$K_{A,B}$ Clave simétrica compartida entre los agentes A y B

N_A Nonce (número aleatorio de un solo uso) generado por el agente A

S Servidor de confianza o KDC (Key Distribution Center)

AST Árbol Sintáctico Abstracto (*Abstract Syntax Tree*)

KDC Centro de Distribución de Claves (*Key Distribution Center*)

MVC Modelo-Vista-Controlador (*Model-View-Controller*)

1

Introducción

1.1 Contexto

En las comunicaciones digitales modernas, la seguridad depende principalmente de que el funcionamiento de los protocolos criptográficos se lleve a cabo rigurosamente. Los protocolos de claves simétricas, encargados de establecer entre los participantes una base de confianza y canales seguros, juegan un papel importante en muchas redes empresariales y de usuarios finales.

A pesar de su importancia, el diseño de este tipo de protocolos es una tarea extremadamente compleja y en ocasiones poco intuitiva. Incluso aquellos algoritmos matemáticamente perfectos pueden no llegar a ser suficientes en entornos de redes hostiles, pues un atacante con control sobre el canal puede poseer la capacidad para interceptar, almacenar o alterar el orden de los mensajes enviados. Por ello, muchos esquemas lógicos que a priori se ven totalmente robustos dejan espacio a vulnerabilidades estructurales casi imperceptibles, como pueden ser ataques de repetición e incluso de suplantación como ocurre con el protocolo Otway-Rees.

Revisar estos protocolos de forma manual tiene como resultado que, debido a la complejidad de muchos de estos, fallos de diseño puedan llegar a pasar inadvertidos por muy exhaustiva que sea la inspección. De esta problemática surge la necesidad de contar con herramientas capaces de analizarlos y detectar las posibles vulnerabilidades. En este contexto, este Trabajo de Fin de Grado tiene como objetivo el desarrollo de un software con la capacidad de facilitar la detección de vulnerabilidades clásicas, automatizando el proceso de evaluación lógica en los protocolos de distribución de claves simétricas.

1.2 Motivación

La mayoría de herramientas existentes centradas en la verificación formal (tal y como pueden ser Scyther, ProVerif o AVISPA [1], a pesar de su gran potencia, presentan interfaces puramente basadas en líneas de comandos o su proceso de aprendizaje es sumamente denso y complejo pues están diseñadas por y para expertos en criptografía matemática. Esto supone una barrera significativa para la iniciación de estudiantes, desarrolladores o analistas de seguridad que buscan auditar el diseño lógico de un protocolo.

Este proyecto viene motivado por la democratización, entendiendo por democratización el hecho de hacer accesible la verificación criptográfica a estudiantes y desarrolladores sin exigir conocimientos avanzados en lógica formal o herramientas de consola complejas. Se trata de promover la conciliación entre la rigurosidad matemática y la usabilidad del software moderno. Mediante la interfaz gráfica interactiva proporcionada, el usuario puede centrarse puramente en la evaluación del protocolo empleando una notación estándar (Alice y Bob), mientras el motor interno abstrae toda la complejidad de la validación sintáctica y la generación del Árbol Sintáctico Abstracto.

Adicionalmente, desde un punto vista académico y personal, el desarrollo de esta herramienta supone un reto formativo de alto valor. Obliga a unificar en un único proyecto completo multitud de conocimientos fundamentales adquiridos a lo largo del Grado en Ingeniería Informática; comprendiendo desde el diseño de persistencia de datos, la ciberseguridad y la arquitectura de software empleando el patrón Modelo-Vista-Controlador, hasta la teoría de Procesadores del Lenguaje y compiladores.

1.3 Objetivos

Objetivo General

Este Trabajo de Fin de Grado está centrado en el diseño e implementación de una aplicación de escritorio orientada a la verificación formal y la detección automática de vulnerabilidades en protocolos de distribución de claves simétricas, empleando la notación criptográfica estándar (Alice y Bob).

Objetivos Específicos

Para alcanzar con éxito la meta general propuesta, se establecen los siguientes objetivos técnicos específicos:

- **Desarrollar un motor de compilación a medida:** Implementar un analizador léxico y un analizador sintáctico de descenso recursivo en Java, que cuenten con la capacidad de procesar la sintaxis de los protocolos y construir un Árbol Sintáctico Abstracto (AST) que facilite su posterior evaluación.
- **Implementar un motor de análisis de vulnerabilidades:** Desarrollar una lógica algorítmica que se encargue de la evaluación del AST bajo las premisas formales del modelo de Dolev-Yao. Este motor tendrá la capacidad de realizar la verificación, de forma automatizada, de la frescura de los mensajes (empleando *nonces*) y la correcta aplicación de las claves simétricas para identificar debilidades estructurales.
- **Diseñar una arquitectura de *software* escalable:** Aplicar el patrón de diseño Modelo-Vista-Controlador (MVC) y el patrón estructural *Composite* para garantizar la independencia entre la lógica y la interfaz.
- **Construir una interfaz gráfica interactiva:** Desarrollar mediante el *framework* JavaFX un entorno visual usable e intuitivo que permita a usuarios no expertos en herramientas formales reducir la curva de aprendizaje.
- **Integrar un sistema de persistencia local seguro:** Implementar una base de datos relacional con SQLite para la gestión y almacenamiento de sesiones de usuario, protocolos y sus vulnerabilidades, asegurando la protección de credenciales mediante funciones como BCrypt, una función de hashes intencionalmente lentos con el fin de mitigar y proteger contra ataques de fuerza bruta.

1.4 Metodología

Para llevar a cabo la planificación y desarrollar este Trabajo de Fin de Grado se ha adoptado un modelo con un ciclo de vida iterativo e incremental, inspirado principalmente por las metodologías ágiles. Se

han llevado a cabo reuniones progresivas con el tutor siguiendo un intervalo no estricto de dos semanas en las que se trataba lo trabajado en el periodo entre la presente y la anterior reunión y se establecía lo que se haría en el siguiente intervalo. Gracias a esta elección ha sido posible tener una adaptación flexible para afrontar las complejidades varias que presenta el motor de análisis, además de facilitar la construcción continua de los demás componentes.

El proceso de ingeniería de *software* se ha estructurado en las siguientes fases principales:

- **Fase de Análisis y Diseño:** En esta fase se realiza la definición exhaustiva de los requisitos funcionales y no funcionales del sistema, se diseña la arquitectura general basada en el patrón Modelo-Vista-Controlador (MVC), se establece el modelado normalizado de la base de datos local y se crean los bocetos (*mockups*) que se usarán para elaborar la interfaz gráfica.
- **Fase de Implementación Modular:** La programación del sistema se dividió en varias iteraciones (*sprints* de desarrollo). Se comenzó con la construcción del núcleo lógico (el Analizador Léxico y el motor de Descenso Recursivo para el AST). Posteriormente se siguió con el desarrollo de la capa de persistencia conectada a SQLite y, finalmente, se llevó a cabo la integración de la capa visual interactiva realizada con JavaFX.
- **Fase de Pruebas y Verificación:** Verificación continua del código mediante la ejecución de casos empíricos. Se comprobó rigurosamente la captura de excepciones ante entradas de texto mal formadas y se verificó la precisión del motor a la hora de detectar vulnerabilidades lógicas bajo las premisas del modelo de Dolev-Yao.

Adicionalmente, con el fin de garantizar la integridad del código fuente, proteger los avances ante posibles pérdidas y llevar un seguimiento ordenado de las versiones del compilador, se ha empleado GitHub como sistema de control de versiones, usando además la herramienta GitHub Desktop para facilitar su uso.

Complementariamente, con el fin de realizar una mejor gestión y seguimiento de las tareas de desarrollo se ha empleado la herramienta **Trello**, un tablero digital basado en la metodología Kanban. Mediante el uso de listas categorizadas (*Backlog*, *Pendientes*, *En progreso* y *Hecho*), se ha mantenido una visión clara del avance del proyecto en todo momento, ayudando a establecer una adecuada prioridad de tareas y a la organización de los *sprints* de desarrollo. Esta herramienta ha resultado especialmente útil para la coordinación de las reuniones periódicas con el tutor, permitiendo visualizar de forma rápida el estado de cada funcionalidad y los objetivos pendientes para cada iteración.

2

Estado del Arte y Marco Teórico

2.1 Notación Criptográfica Estándar

En la literatura académica sobre ciberseguridad, la descripción de los protocolos criptográficos suele resultar sumamente densa. Para mitigar este problema y facilitar la comprensión del flujo de mensajes, se ha adoptado de forma estandarizada un repertorio de personajes ficticios. Esta convención permite abstraer la complejidad técnica y modelar las comunicaciones en un formato legible para el ser humano.

Los actores principales que conforman esta notación, y que representan la base estructural sintáctica del analizador desarrollado en este proyecto, son los siguientes:

- **Alice (A) y Bob (B):** Representan a los participantes principales de la comunicación. Alice es la entidad que desea establecer un canal seguro, mientras que Bob es el receptor legítimo de dicha petición.
- **Trent o KDC (Centro de Distribución de Claves):** Actúa como la tercera parte de confianza (*Trusted Third Party*). Su rol es fundamental en los protocolos de claves simétricas, pues al compartir una clave secreta individualizada tanto con Alice como con Bob a largo plazo, es el encargado de generar y distribuir las claves de sesión que permiten a los otros participantes establecer conexiones seguras.
- **Eve o Mallory (El Atacante):** Representan las entidades hostiles existentes dentro de la red.

Eve (*eavesdropper*) suele representar a un atacante pasivo que se limita a escuchar el canal, mientras que Mallory representa a un atacante activo capaz de interceptar, modificar y suplantar mensajes.

Para representar la transmisión de información entre estos agentes, se emplea una sintaxis direccional estandarizada. Una sentencia típica sigue el formato **Emisor -> Receptor : Mensaje**. De este modo, si Alice envía a Bob un identificador aleatorio (*nonce*) cifrado mediante una clave simétrica compartida, la transcripción formal se expresaría de la siguiente manera:

$$\text{Alice} \rightarrow \text{Bob} : \{ \text{Nonce_A} \}_{K_{ab}}$$

Esta gramática estructurada es exactamente la que el *software* propuesto en este Trabajo de Fin de Grado exige como entrada de datos en su editor de texto para poder proceder a la generación del Árbol Sintáctico Abstracto y su posterior evaluación automatizada.

2.2 Conceptos Criptográficos

Se debe establecer una sólida base teórica sobre los propios mecanismos de seguridad que componen los protocolos analizados, esto para así poder comprender la naturaleza de las vulnerabilidades que este sistema trata de detectar. Este Trabajo de Fin de Grado se centra en los siguientes conceptos fundamentales para ello.

2.2.1 Criptografía Simétrica

La criptografía simétrica es un paradigma de la seguridad en el que los interlocutores (sean Alice y Bob) emplean una única clave secreta compartida (K) tanto para cifrar el texto plano que enviarán como para descifrar el criptograma recibido. Destaca principalmente debido a su eficiencia computacional. Sin embargo, presenta una debilidad intrínseca a la distribución de claves: para que Alice y Bob puedan comunicarse de forma segura, es necesario haber intercambiado la clave K a través de un canal seguro previo. Es precisamente esta problemática la que intentan resolver aquellos protocolos que cuentan con un centro de distribución de claves (KDC). En el contexto de este proyecto, se asume que los algoritmos de cifrado empleados son matemáticamente perfectos, centrando el análisis exclusivamente en los posibles fallos lógicos que puedan sucederse durante el intercambio de dichas claves.

2.2.2 Nonces (Number used ONCE)

Un *nonce* es un valor numérico pseudoaleatorio e impredecible generado con el fin de emplearse una única vez durante una sesión de comunicación específica. Empleando notación estándar, es representado típicamente vinculado a su creador, como N_A (*nonce* generado por Alice). Está pensado principalmente no para ocultar información confidencial a ojos de posibles atacantes, sino con el fin de actuar como una marca empleada para vincular inequívocamente un mensaje con su correspondiente respuesta cifrada.

2.2.3 Frescura de los Mensajes

Llamamos *frescura* (*freshness*) a la propiedad lógica que garantiza la reciente generación de un paquete recibido en el contexto de la sesión actual, y así evitar que sea una grabación o copia de una comunicación pasada. Para garantizarla es necesaria la correcta inclusión y verificación de los ya tratados *nonces*. En el caso de producirse un fallo a la hora de confirmar correctamente esta propiedad, se desencadena una brecha de seguridad que deja la comunicación expuesta a ataques de repetición (*Replay Attacks*). En estos escenarios, si se da el caso de que exista un intruso que haya almacenado el tráfico de red, este podría retransmitir mensajes antiguos para tratar de suplantar a las entidades interlocutoras o provocar que se tuvieran que volver a generar aquellas claves que hayan quedado comprometidas.

2.3 El Modelo del Intruso de Dolev-Yao

El modelo del intruso de Dolev-Yao [2] es el marco teórico formal en el que se basa el motor de análisis de este proyecto. Propuesto por Danny Dolev y Andrew Yao en 1983, establece las premisas bajo las cuales se estudia la seguridad de los protocolos criptográficos de comunicación de forma rigurosa.

La hipótesis fundamental de este modelo asume que los algoritmos de cifrado empleados son matemáticamente perfectos. En consecuencia, el análisis de seguridad no se centra en el algoritmo en sí, sino en las posibles debilidades lógicas del flujo de intercambio de mensajes. Básicamente, el análisis de seguridad propuesto lo que realmente evalúa es el protocolo y los mensajes que este intercambia.

Bajo este modelo, el **intruso** (denominado convencionalmente Mallory o I) no es un observador

pasivo, sino una entidad activa con control total sobre el canal de comunicación. Sus capacidades formales comprenden:

- **Interceptar** cualquier mensaje en tránsito entre los agentes legítimos.
- **Almacenar** mensajes capturados para reutilizarlos en sesiones futuras (lo que posibilita los ataques de repetición o *Replay Attacks*).
- **Inyectar** mensajes nuevos en el canal, haciéndose pasar por cualquier agente (*Spoofing*).
- **Reordenar o suprimir** mensajes en tránsito.
- **Descifrar** únicamente aquellos criptogramas para los cuales posee la clave correspondiente (principio de opacidad criptográfica).

Este modelo es universalmente adoptado por las herramientas de verificación formal de protocolos criptográficos [1, 6]. Su relevancia en este proyecto reside en que las reglas de auditoría del **MotorAnálisis** están directamente inspiradas en estas premisas: cada verificación de frescura, de desafío-respuesta o de accesibilidad de claves simula el conocimiento que un intruso de Dolev-Yao podría acumular a lo largo del intercambio de mensajes.

2.4 Teoría de Compiladores

Para desarrollar el corazón de la aplicación, el motor de análisis, ha sido necesaria la puesta en práctica de conceptos referentes a la Teoría de Procesadores del Lenguaje. Para ser capaz de evaluar lógicamente un protocolo como debe hacerlo el *software* desarrollado, el sistema debe poder comprender y estructurar un texto plano que el usuario introduzca en el editor. Este proceso se divide en dos fases fundamentales:

2.4.1 Análisis Léxico

La primera fase del proceso de compilación es el análisis léxico. El objetivo de esta es leer, del código fuente, una secuencia de caracteres (el protocolo una vez transcrito) y agruparlos en unidades lógicas con significado (como agentes o mensajes), denominadas *tokens*. En cuanto a la implementación, se han usado Expresiones Regulares (Regex) para el diseño del analizador léxico. Estas, actuando

como patrones de reconocimiento, permiten verificar que cada línea introducida presente la estructura sintáctica esperada, descartando malformaciones y errores tipográficos previos a permitir que se someta el texto a proceso de mayor coste computacional.

2.4.2 Análisis Sintáctico y Árbol Sintáctico Abstracto (AST)

Tras superar la validación léxica, se pasa a la segunda fase, el análisis sintáctico. Esta etapa se centra en comprobar que las expresiones formadas a partir de la secuencia de *tokens* sean válidas y en que esa información sea organizada siguiendo un esquema jerárquico en la memoria del sistema, requisito necesario para que sea posible analizar las vulnerabilidades de forma correcta. Con ese objetivo, se emplea un analizador sintáctico de descenso recursivo (*Recursive Descent Parser*), empleando un enfoque *top-down* que evalúa las sentencias partiendo desde lo más general hasta separar los operandos más concretos.

Este análisis resulta en la generación en memoria de un Árbol Sintáctico Abstracto (AST, por sus siglas en inglés). Este consiste en una serie de datos representados en forma de árbol capturando la estructura del código de entrada a la vez que detalles como espacios o puntuaciones no esenciales quedan omitidos.

El AST ha sido modelado, en la estructura lógica del proyecto, aplicando el patrón de diseño *Composite*. Lo cual permite que tanto los nodos hoja individuales (agentes o *nonce*) como nodos compuestos (bloque de datos cifrados que contiene varios elementos) sean tratados uniforme y equitativamente. Gracias a ello, el motor puede recorrer el árbol recursivamente, lo que permitirá solucionar la problemática de la encapsulación múltiple de mensajes en criptografía (por ejemplo: al enviar Alice a Bob un bloque cifrado, que a su vez podría albergar identificadores además de sub-bloques cifrados), permitiendo así la evaluación automática de cualquier nivel de anidamiento criptográfico.

Para ilustrar gráficamente esta jerarquía de objetos y la implementación del patrón *Composite*, el diagrama de clases completo correspondiente al analizador sintáctico y a la estructura del AST puede consultarse en el **Anexo C**.

2.5 Herramientas Existentes

Podemos encontrar varias herramientas ampliamente utilizadas por la comunidad científica y académica que se podrían clasificar en el mismo ámbito que la aplicación de este proyecto: la verificación formal de protocolos criptográficos. Algunas de las más destacadas son AVISPA [1] (*Automated Validation of Internet Security Protocols and Applications*) y Scyther. Estas poseen la capacidad de modelar escenarios complejos y detectar vulnerabilidades bajo las premisas del modelo de Dolev-Yao, al igual que en el sistema propuesto.

A pesar de sus capacidades, estas herramientas presentan problemas que pueden suponer una barrera a la hora de iniciarse con ellas. En su mayoría, emplean lenguajes de especificación muy complejos como pueden ser HLPSL (*High Level Protocol Specification Language*) en el caso de AVISPA, o el lenguaje *spdl* que usa Scyther. Además, para interactuar con estas aplicaciones, la mayoría de las veces es necesario emplear una interfaz de línea de comandos (CLI) o en caso de contar con entornos gráficos, estos suelen ser poco intuitivos para usuarios no expertos.

Este Trabajo de Fin de Grado tiene la capacidad de suplir esas carencias. La aplicación desarrollada no tiene como objetivo la competencia en cuanto a poder bruto con herramientas de ese nivel, sino encontrar su nicho supliendo la falta de accesibilidad de las mismas. Al proporcionar un entorno gráfico interactivo, para el que se ha empleado JavaFX, sumado a que se permite el uso de la notación criptográfica estándar (Alice y Bob), permite reducir la curva de aprendizaje para aquellos que deseen iniciarse en la evaluación de protocolos, siendo además una alternativa excelente para fines didácticos y educativos.

3

Análisis del Sistema

Aquí se encuentra detallado el análisis de requisitos y la especificación funcional del *software* propuesto. El objetivo principal de esta fase es definir de manera clara, estructurada y sin ambigüedades aquello que debe hacer el sistema, proporcionando las bases que serán usadas para construir la arquitectura y la interfaz en las fases posteriores.

3.1 Requisitos Funcionales

Los requisitos funcionales definen los comportamientos y servicios específicos que el sistema ha de proporcionar, describiendo la interacción técnica entre la aplicación y sus usuarios. A continuación, se enumeran categorizados por módulos operativos:

3.1.1 Requisitos de registro y cuentas de usuario

RF1.1 - Registrar las credenciales del usuario. El sistema debe permitir al usuario introducir unas credenciales para su registro. Para verificar la propiedad de la cuenta y evitar suplantaciones, el sistema generará un código numérico aleatorio de 6 dígitos y lo enviará al correo electrónico del usuario usando el protocolo SMTP. Para completar el registro en la base de datos el usuario deberá introducir dicho código a través de la interfaz. *Prioridad Alta.*

RF1.2 - Iniciar sesión en la aplicación. El sistema debe permitir al usuario emplear las credenciales usadas anteriormente para registrarse para poder iniciar sesión en la aplicación y acceder a los datos de su cuenta. *Prioridad Alta.*

RF1.3 - Cerrar sesión. La aplicación debe contar con un botón de fácil acceso que permitirá al

usuario cerrar la sesión activa de forma segura. *Prioridad Media.*

RF1.4 - Gestionar cuenta. El sistema debe permitir acceder al usuario a una interfaz para hacer cambios en los parámetros de su cuenta. *Prioridad Baja.*

RF1.5 - Cambiar contraseña. El sistema debe contar con la posibilidad de permitir al usuario cambiar la contraseña de su cuenta a través de la pantalla de gestión de cuenta. *Prioridad Baja.*

RF1.6 - Eliminar la cuenta. El sistema debe permitir al usuario eliminar permanentemente su cuenta, borrando así sus datos de la base de datos a través de la pantalla de gestión de cuenta. *Prioridad Baja.*

RF1.7 - Comprobar contraseña. El sistema debe comprobar si la contraseña introducida por el usuario al iniciar sesión es correcta. *Prioridad Alta.*

RF1.8 - Recuperar contraseña. El usuario, en caso de olvidar su clave, debe poder introducir su correo electrónico para solicitar un restablecimiento. El sistema generará y enviará un código de verificación de 6 dígitos al correo. Tras validar el código el sistema mostrará al usuario una pestaña para introducir la nueva contraseña. *Prioridad Baja.*

3.1.2 Requisitos del dashboard

RF2.1 - Mostrar Dashboard. El sistema, tras haber iniciado sesión, mostrará un panel con diversos datos. *Prioridad Alta.*

RF2.2 - Mostrar las estadísticas globales. El sistema debe mostrar en la pantalla principal, a través de un panel, datos tales como el número de protocolos analizados y la cantidad total de vulnerabilidades detectadas. *Prioridad Alta.*

RF2.3 - Desglosar las vulnerabilidades. El sistema muestra un panel donde se debe mostrar en pantalla la cantidad de veces que se ha detectado cada tipo de fallo lógico en los protocolos del usuario. *Prioridad Media.*

RF2.4 - Acceso a los protocolos recientes. La aplicación debe tener listados los últimos protocolos guardados o analizados a los que se ha accedido, permitiendo acceder a estos con un solo click. *Prioridad Media.*

RF2.5 - Renombrar los protocolos guardados. El sistema, a través del panel de control, debe permitir al usuario poder renombrar los protocolos guardados en el historial. *Prioridad Baja.*

RF2.6 - Eliminar los protocolos guardados. El sistema, a través del panel de control, debe permitir al usuario eliminar los protocolos guardados en el historial. *Prioridad Baja.*

RF2.7 - Cargar protocolo desde el historial. El sistema debe permitir al usuario cargar en el editor los protocolos visibles en el listado de historial. El sistema deberá validar que el archivo en el path de la base de datos sigue existiendo en local; en caso de haber sido borrado o movido por el usuario, se mostrará una advertencia y se ofrecerá la posibilidad de eliminar el registro. *Prioridad Media.*

3.1.3 Requisitos de archivo de protocolo

RF3.1 - Escribir los protocolos. El sistema ha de proporcionar un área de texto donde el usuario podrá introducir la especificación del protocolo a evaluar. El sistema será capaz de reconocer diferentes mensajes al separarlos mediante un salto de línea. *Prioridad Alta.*

RF3.2 - Limpiar editor de texto. El sistema da la posibilidad al usuario mediante un botón de eliminar todo el contenido presente en la interfaz de notación. *Prioridad Baja.*

RF3.3 - Numeración de líneas. El sistema debe enumerar las líneas del editor de texto para que el usuario pueda ubicar rápidamente dónde el motor de análisis ha detectado una vulnerabilidad. *Prioridad Baja.*

RF4 - Definir los agentes participantes. El sistema debe permitir al usuario declarar los actores que participan en la comunicación. *Prioridad Alta.*

RF5 - Identificar el texto cifrado. El sistema debe ser capaz de identificar qué parte de cada mensaje viaja como texto plano y qué parte va encapsulada bajo alguna clave de cifrado simétrico. *Prioridad Alta.*

RF6.1 - Gestionar nonces y marcas de tiempo. El sistema debe reconocer la declaración de variables aleatorias y marcas de tiempo dentro de los mensajes para evaluar correctamente la frescura (Freshness) de la comunicación. *Prioridad Alta.*

RF6.2 - Cumplir Desafío-Respuesta. El sistema debe rastrear si un nonce enviado por uno de los agentes en un paso es devuelto a este por el otro agente. *Prioridad Alta.*

3.1.4 Requisitos de análisis y notificación de vulnerabilidades

RF7.1 - Validar la sintaxis previamente. Al pulsar el botón de análisis, antes de ejecutar el motor, el sistema debe comprobar que la sintaxis en cada una de las líneas es correcta. Si hay un error, el sistema debe notificarlo para su solución. *Prioridad Alta.*

RF7.2 - Ejecutar el motor de análisis. El sistema, una vez validado el léxico, procede a evaluar la secuencia completa de mensajes en busca de cualquier vulnerabilidad. *Prioridad Alta.*

RF7.3 - Identificar ataques de suplantación. El sistema debe ser capaz de identificar si un protocolo es vulnerable a que un atacante suplante la identidad de uno de los agentes. *Prioridad Alta.*

RF7.4 - Confirmar que el protocolo es seguro. Si no se encuentra ninguna brecha, el sistema debe mostrar por pantalla la confirmación de que el protocolo es totalmente seguro. *Prioridad Alta.*

RF7.5 - Mostrar detalladamente las vulnerabilidades. Al finalizar el análisis, si se detectan fallos, el sistema indicará los nombres de las vulnerabilidades, la línea exacta y una breve descripción. *Prioridad Alta.*

RF7.11 - Identificar falta de autenticación mutua. El motor debe ser capaz de detectar si una clave de sesión distribuida es utilizada activamente por ambas partes. Si un agente recibe una clave pero nunca la utiliza para cifrar y devolver una respuesta, el sistema debe alertar de una autenticación unilateral o falta de confirmación. *Prioridad Alta.*

RF7.12 - Identificar ataques de repetición (Replay Attacks). El sistema debe detectar vulnerabilidades de repetición cuando se envían tickets o claves de sesión sin garantías de frescura (ausencia de nonces creados por el receptor o marcas de tiempo), alertando de que un atacante podría reinyectar sesiones caducadas. *Prioridad Alta.*

3.1.5 Requisitos de importación y exportación

RF8.1 – Exportar protocolos en archivos. El sistema debe permitir exportar el protocolo que se está editando a un archivo local. *Prioridad Media.*

RF8.2 – Importar protocolos desde un archivo. El sistema debe permitir cargar un protocolo desde un archivo al editor, restaurando automáticamente las líneas. *Prioridad Media.*

RF8.3 – Autoguardado y sobrescritura de protocolos existentes. Al invocar el análisis sobre un protocolo previamente guardado en el equipo, el sistema debe sobrescribir automáticamente el archivo local con el contenido actual del editor antes de procesarlo, previniendo así la pérdida de datos y manteniendo el historial actualizado. *Prioridad Media.*

3.1.6 Requisitos de generación de informes y soporte

RF9 – Generar el informe del análisis. El sistema debe compilar los resultados del último protocolo analizado en un informe estructurado y permitir su exportación en formato PDF. *Prioridad Media.*

RF10 – Mostrar guía de aprendizaje integrada. El sistema dispondrá de un botón de ayuda que desplegará información esencial sobre sintaxis y funciones básicas. *Prioridad Baja.*

3.1.7 Requisitos de base de datos

RF11 – Exigir confirmación para acciones irreversibles. El sistema debe requerir una confirmación explícita mediante ventana emergente antes de eliminar cuentas o protocolos. *Prioridad Media.*

RF12 – Mantener persistencia del historial. El sistema debe registrar cada análisis en la base de datos local almacenando exclusivamente los metadatos del protocolo (ID, nombre, fecha, veredicto y ruta local del archivo), el usuario y la vulnerabilidad. *Prioridad Alta.*

3.2 Requisitos No Funcionales

Los requisitos no funcionales establecen las restricciones y los criterios de calidad que deberá cumplir el sistema, abarcando aspectos como el rendimiento, la seguridad de la arquitectura y la experiencia de usuario.

3.2.1 Rendimiento y robustez

RNF1 - Rendimiento del motor de análisis. La aplicación debe procesar los protocolos rápidamente.

El tiempo de ejecución desde que se pulsa el botón hasta obtener el resultado no debe ser mayor a 4 segundos.

RNF2 - Robustez del analizador sintáctico. El sistema no debe cerrarse repentinamente ante entradas de texto mal formadas; debe capturar las excepciones y mostrar un mensaje de error.

3.2.2 Usabilidad

RNF3 - Accesibilidad de las funciones. Cualquier funcionalidad principal debe ser alcanzable en un máximo de 4 clics desde la interfaz principal.

RNF4 - Cobertura de la guía integrada. La ayuda debe ser de fácil lectura y explicar el acceso a las funciones esenciales. Se entregará un cuestionario para que un grupo de usuarios valide empíricamente este requisito.

3.2.3 Seguridad y arquitectura

RNF5 - Cifrado de credenciales locales. Las contraseñas en la base de datos deben estar cifradas. Se considerará un fallo si alguna contraseña se almacena en texto plano.

RNF6 - Escalabilidad de vulnerabilidades. El código debe estar diseñado para añadir nuevos tipos de ataques fácilmente modificando solo el paquete lógico, sin alterar la interfaz (JavaFX) ni la base de datos. Se validará mediante inspección de código y cumplimiento del patrón MVC.

3.3 Casos de Uso

En este apartado, teniendo como base los requisitos previamente citados, será modelado el comportamiento del mismo sistema a través de los Casos de Uso. Estos permiten describir la propia aplicación desde el punto de vista de la interacción con el usuario, abstrayendo los detalles de la implementación.

3.3.1 Identificación de Actores

En el contexto de esta aplicación, dada su naturaleza de herramienta de escritorio con apoyo de red para autenticación, se han identificado los siguientes actores:

- **Usuario no autenticado:** Representa a cualquier individuo que interactúa con la aplicación antes de iniciar sesión. Sus acciones están limitadas al registro, recuperación de contraseña y autenticación.
- **Usuario autenticado:** Actor principal del sistema. Representa al estudiante, investigador o profesional que, tras validar sus credenciales, interactúa de forma completa con la interfaz gráfica para el análisis y gestión de protocolos.
- **Servidor SMTP (Actor Secundario):** Actor automatizado externo. El sistema interactúa con él de forma unidireccional para delegar el envío de correos electrónicos con los códigos numéricos de verificación.

3.3.2 Diagrama de Casos de Uso

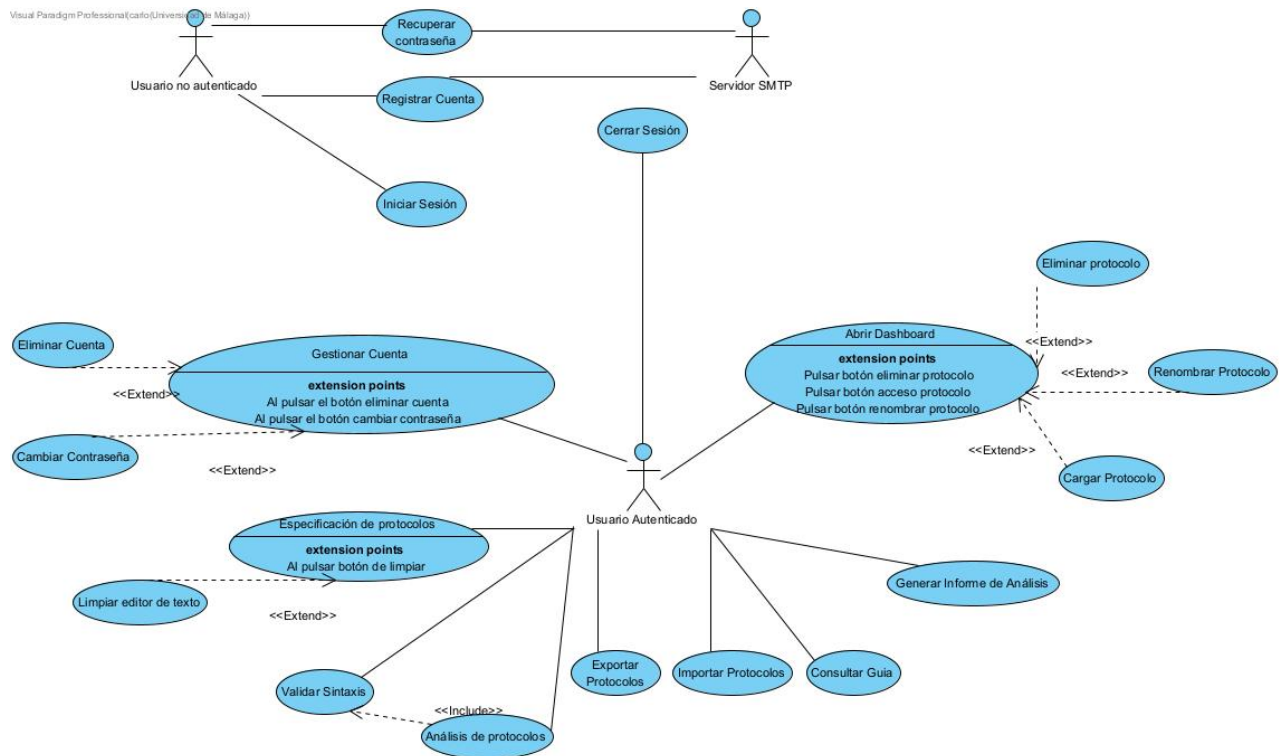


Figura 3.1. Diagrama general de Casos de Uso del sistema.

3.3.3 Especificación Detallada de Casos de Uso

A continuación se detallan los flujos de ejecución y las condiciones de cada uno de los casos de uso identificados en el sistema. Cabe destacar que el diagrama de secuencia correspondiente al proceso central de la aplicación (Análisis de protocolos) se incluye en este mismo capítulo para facilitar su comprensión inmediata. Sin embargo, por motivos de extensión y para mantener la legibilidad del documento, los demás diagramas de secuencia (registro de usuarios, autenticación, importación, generación de informes e inicio de sesión) han sido desplazados al **Anexo A**.

3.3.3.1 Actor: Usuario no autenticado

CU01 - Registrar cuenta

Código único: CU01

Nombre: Registrar cuenta

Precondición: El usuario no está autenticado y tiene conexión a la red (para el Servidor SMTP) y a

la base de datos local.

Escenario principal:

1. El actor pulsa el botón de registro.
2. El sistema muestra el formulario de registro.
3. El actor introduce su correo electrónico (nombre de usuario) y contraseña.
4. El sistema valida que el correo no exista previamente en la BBDD.
5. El sistema genera un código de verificación aleatorio de 6 dígitos.
6. El sistema delega en el Servidor SMTP el envío de un correo electrónico con el código al usuario mediante un hilo de ejecución secundario (asíncrono) para no bloquear la interfaz.
7. El sistema actualiza la pantalla solicitando la introducción de dicho código numérico.
8. El actor introduce el código recibido en su bandeja de entrada.
9. El sistema valida el código, delega en `GestorUsuarios` la encriptación de la clave mediante hash BCrypt y la guarda en la BBDD local.
10. El sistema muestra un mensaje de éxito y redirige al inicio de sesión.

Escenario alternativo:

- **4a.** El correo electrónico ya existe en la BBDD.
- **4b.** El sistema muestra un mensaje de error advirtiendo de la duplicidad.
- **4c.** El sistema limpia el formulario y vuelve al paso 2.
- **8a.** El código introducido es incorrecto.
- **8b.** El sistema muestra un mensaje de error y permite al actor volver a introducirlo.

Postcondición: El sistema crea un nuevo registro de usuario verificado y válido para iniciar sesión.

**El diagrama de secuencia de este caso de uso puede consultarse en la Figura [A.1](#) del Anexo A.*

CU02 - Iniciar sesión

Código único: CU02

Nombre: Iniciar sesión

Precondición: El usuario no está autenticado y posee una cuenta registrada.

Escenario principal:

1. El sistema muestra el formulario de inicio de sesión.
2. El actor introduce sus credenciales (usuario y contraseña).
3. El sistema delega en el `GestorUsuarios` la autenticación.
4. El `GestorUsuarios` consulta la base de datos y verifica que el hash de la contraseña coincide mediante el algoritmo BCrypt.
5. El sistema registra la sesión activa en el componente *Singleton* `GestorSesion`.
6. El sistema muestra el Dashboard principal.

Escenario alternativo:

- **4a.** El usuario no existe o la contraseña es incorrecta.
- **4b.** El sistema muestra un mensaje de error de "Credenciales inválidas".
- **4c.** El sistema vuelve al paso 1.

Postcondición: El usuario accede a la aplicación autenticado como "Usuario autenticado".

**El diagrama de secuencia de este caso de uso puede consultarse en la Figura A.2 del Anexo A.*

CU03 – Recuperar contraseña

Código único: CU03

Nombre: Recuperar contraseña

Precondición: El usuario no ha iniciado sesión y el equipo cuenta con conexión a la red (Servidor SMTP).

Escenario principal:

1. El actor selecciona la opción de recuperar contraseña en la pantalla de inicio de sesión.
2. El sistema solicita la dirección de correo electrónico asociada a la cuenta.
3. El actor introduce el correo y confirma la acción.

4. El sistema consulta la BBDD y verifica que el correo existe.
5. El sistema genera un código de verificación aleatorio de 6 dígitos.
6. El sistema delega en el Servidor SMTP el envío asíncrono del correo electrónico con el código.
7. El sistema solicita la introducción de dicho código numérico por pantalla.
8. El actor introduce el código recibido.
9. El sistema valida el código y muestra los campos para establecer y confirmar la nueva contraseña.
10. El actor teclea su nueva contraseña en ambos campos.
11. El sistema cifra la nueva contraseña, sobrescribe la antigua en la BBDD local y notifica el éxito.

Escenario alternativo:

- **4a.** El correo no está registrado en el sistema.
- **4b.** El sistema muestra un error y devuelve al actor al paso 2.
- **8a.** El código introducido es incorrecto.
- **8b.** El sistema muestra un error y permite reintentarlo.
- **10a.** Las contraseñas introducidas no coinciden.
- **10b.** El sistema advierte del error y limpia los campos para su reintroducción.

Postcondición: La cuenta se actualiza con la nueva clave y el sistema redirige al inicio de sesión.

3.3.3.2 Actor: Usuario autenticado

CU04 - Cerrar sesión

Código único: CU04

Nombre: Cerrar sesión

Precondición: El usuario se encuentra autenticado en el sistema.

Escenario principal:

1. El actor pulsa el botón de cerrar sesión desde cualquier pantalla.

2. El sistema muestra un cuadro de dialogo pidiendo confirmación
3. El actor confirma que desea cerrar sesión.
4. El sistema destruye los datos de la sesión activa en memoria.
5. El sistema redirige al usuario a la pantalla de inicio (inicio sesión/registro).

Escenario alternativo:

- **2a.** El actor pulsa el botón de cancelar en la pestaña de confirmación.
- **2b.** El sistema muestra cierra pestaña de confirmación.

Postcondición: El usuario pierde el acceso a sus datos y pasa a ser un usuario no autenticado.

CU05 - Gestionar cuenta

Código único: CU05

Nombre: Gestionar cuenta

Precondición: El usuario está autenticado.

Escenario principal:

1. El actor accede a los ajustes de su perfil mediante un botón.
2. El sistema recupera los metadatos de la cuenta activa.
3. El sistema muestra la pantalla con las opciones de cuenta (cambiar clave, eliminar cuenta).

Escenario alternativo: No aplica.

Postcondición: El usuario visualiza la interfaz de gestión de su perfil.

CU06 - Eliminar cuenta

Código único: CU06

Nombre: Eliminar cuenta

Precondición: El usuario está en la pantalla de gestión de cuenta.

Escenario principal:

1. El actor pulsa el botón de eliminar cuenta.
2. El sistema despliega un cuadro de diálogo pidiendo confirmación de seguridad.
3. El actor confirma la eliminación.
4. El sistema borra en cascada todos los protocolos y estadísticas del historial de ese usuario.
5. El sistema borra las credenciales del usuario de la BBDD.
6. El sistema fuerza el cierre de sesión y redirige al inicio.

Escenario alternativo:

- **3a.** El actor cancela la operación en el cuadro de diálogo.
- **3b.** El sistema cierra el aviso y cancela el borrado, volviendo a la pantalla de gestión.

Postcondición: Todos los datos y el rastro del usuario son eliminados permanentemente del sistema local.

**El diagrama de secuencia de este caso de uso puede consultarse en la Figura A.3 del Anexo A.*

CU07 - Cambiar contraseña

Código único: CU07

Nombre: Cambiar contraseña

Precondición: El usuario está en la pantalla de gestión de cuenta.

Escenario principal:

1. El actor pulsa el botón de cambiar contraseña.
2. El sistema muestra un formulario pidiendo la clave actual y la nueva.
3. El actor introduce los datos.
4. El sistema verifica que la clave actual coincide con la guardada en la BBDD.
5. El sistema cifra la clave nueva y actualiza el registro en la BBDD.
6. El sistema muestra un mensaje de actualización exitosa.

Escenario alternativo:

- **4a.** La contraseña actual introducida no es válida.
- **4b.** El sistema deniega el cambio y muestra un error.
- **4c.** El sistema vuelve al paso 2.

Postcondición: Las credenciales del usuario se han actualizado y las antiguas dejan de tener validez.

CU08 - Especificación de protocolos

Código único: CU08

Nombre: Especificación de protocolos

Precondición: El usuario está autenticado y en la pantalla del editor.

Escenario principal:

1. El actor pulsa en crear un nuevo protocolo o importa uno desde un archivo.
2. El actor define los nombres de los agentes participantes (ej. Alice, Bob).
3. El actor interactúa con el área de texto o los paneles de configuración.
4. El actor escribe secuencialmente las sentencias criptográficas (mensajes, nonces, claves).

Escenario alternativo: El actor en lugar de escribir vuelve al dashboard.

Postcondición: El modelo del protocolo queda listo para procesarse.

CU09 - Limpieza del editor de texto

Código único: CU09

Nombre: Limpieza del editor de texto

Precondición: El usuario tiene información (texto o agentes) cargada en el editor.

Escenario principal:

1. El actor pulsa el botón de limpieza de editor.
2. El sistema pide confirmación para evitar borrados accidentales.
3. El actor confirma la operación.

4. El sistema vacía el área de texto y reinicia las variables de la memoria visual.

Escenario alternativo:

- **3a.** El actor rechaza la limpieza.
- **3b.** El sistema aborta la limpieza y mantiene el texto intacto.

Postcondición: El área de trabajo queda restablecida a su estado inicial en blanco.

CU10A - Validar sintaxis del protocolo

Código único: CU10A

Nombre: Validar sintaxis del protocolo

Precondición: Hay al menos una línea de texto en el editor de protocolos.

Escenario principal:

1. El actor pulsa el botón de validar sintaxis.
2. El controlador delega en `ServicioAnálisis` la invocación del `AnalizadorLexico`.
3. El `AnalizadorLexico` aplica el patrón de expresión regular sobre cada línea del editor de forma secuencial.
4. Si todas las líneas superan la validación estructural, el sistema muestra un mensaje de confirmación indicando que la sintaxis es correcta.

Escenarios alternativos:

- **3a.** Una línea no respeta la estructura `Emisor -> Receptor : Contenido`.
- **3b.** El sistema interrumpe la validación y lanza una `AnalizadorException` controlada.
- **3c.** El controlador captura la excepción y muestra al usuario un cuadro de diálogo indicando el número de línea y el tipo de error sintáctico detectado.

Postcondición: El usuario conoce si la especificación del protocolo es sintácticamente válida antes de lanzar el análisis completo.

CU10B – Análisis de protocolo

Código único: CU10B

Nombre: Análisis de protocolo

Precondición: Hay un protocolo especificado en el editor de texto.

Escenario principal:

1. El actor pulsa el botón de analizar protocolo.
2. El controlador delega el flujo en la fachada **ServicioAnálisis**.
3. El **AnalizadorLexico** comprueba la correcta sintaxis de las líneas (paréntesis, flechas, agentes).
4. El **MotorAnálisis** evalúa el flujo en busca de vulnerabilidades (desafío-respuesta, frescura, etc).
5. El **ServicioAnálisis** ordena el autoguardado en disco y guarda el veredicto en la BBDD.
6. El sistema muestra por pantalla una notificación indicando el resultado del análisis de seguridad.

Escenarios alternativos:

- **3a.** (Error sintáctico) Hay un error tipográfico en el código.
- **3b.** El sistema interrumpe el análisis antes del paso 4.
- **3c.** El sistema notifica al usuario indicando el fallo de escritura.
- **4a.** (Vulnerabilidad detectada) El motor detecta un fallo lógico (ej. ataque de reenvío).
- **4b.** El sistema guarda el veredicto negativo y la ruta del archivo en la BBDD.
- **4c.** El sistema muestra una alerta roja con la vulnerabilidad, la línea exacta y la descripción.

Postcondición: Se ha emitido un juicio técnico sobre el protocolo y sus metadatos se han guardado en las métricas de la cuenta.

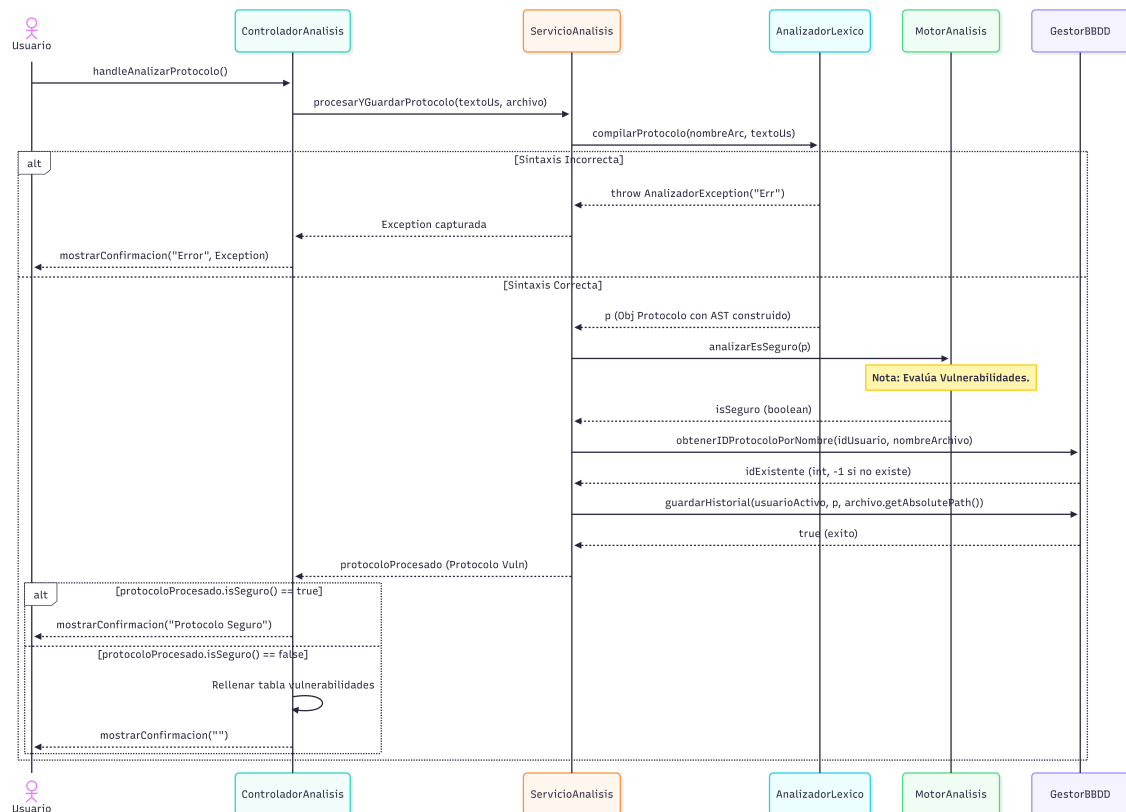


Figura 3.2. Diagrama de secuencia del Caso de Uso 10: Ejecución del motor criptográfico y evaluación de vulnerabilidades.

CU11 - Guardar protocolo

Código único: CU11

Nombre: Guardar protocolo

Precondición: Hay un protocolo definido en el editor de texto.

Escenario principal:

1. El actor selecciona la opción de guardar protocolo.
2. El sistema abre la ventana nativa de guardado del sistema operativo.
3. El actor elige la ruta y el nombre del archivo.
4. El sistema toma el texto y configuración y lo guarda como un archivo local estructurado (ej. formato .prot).
5. El sistema notifica que el guardado ha terminado.

Escenario alternativo:

- **3a.** El actor cancela la ventana de diálogo del sistema operativo.
- **3b.** El sistema aborta la operación de guardado.

Postcondición: Existe una copia física del modelo guardada en el disco duro del usuario.

CU12 – Importar protocolo

Código único: CU12

Nombre: Importar protocolo

Precondición: El usuario está en la interfaz de especificación de la aplicación.

Escenario principal:

1. El actor pulsa el botón de importar protocolo.
2. El sistema abre el buscador de archivos del sistema operativo.
3. El actor selecciona un archivo compatible.
4. El sistema lee el contenido, lo parsea y restaura las variables.
5. El sistema carga automáticamente las sentencias en el área de texto.

Escenario alternativo:

- **4a.** El archivo está corrupto o no tiene un formato soportado.
- **4b.** El sistema cancela la carga y muestra un error de importación por pantalla.

Postcondición: El protocolo proveniente del archivo reemplaza el contenido actual del editor.

**El diagrama de secuencia de este caso de uso puede consultarse en la Figura A.4 del Anexo A.*

CU13 – Consultar guía

Código único: CU13

Nombre: Consultar guía

Precondición: El usuario tiene la aplicación abierta.

Escenario principal:

1. El actor hace clic en el botón de ayuda/guía.
2. El sistema recupera los textos estáticos documentales.
3. El sistema despliega un panel lateral o ventana emergente mostrando las instrucciones básicas, sintaxis y uso.

Escenario alternativo: No aplica.

Postcondición: El usuario obtiene soporte informativo sin interrumpir su flujo de trabajo.

CU14 - Generar informe del análisis

Código único: CU14

Nombre: Generar informe de análisis

Precondición: Se ha ejecutado el análisis de un protocolo recientemente.

Escenario principal:

1. El actor pulsa la opción de generar informe de análisis.
2. El sistema recopila el texto del protocolo en memoria y el veredicto del último análisis.
3. El sistema estructura estos datos en un formato documental.
4. El sistema pide al usuario una ubicación de guardado local.
5. El informe se exporta al disco duro en formato .pdf.

Escenario alternativo:

- **2a.** No existe un análisis previo en la sesión actual.
- **2b.** El sistema avisa al usuario de que primero debe analizar un protocolo para poder reportarlo.

Postcondición: El usuario obtiene un documento resumido del análisis para su uso externo.

**El diagrama de secuencia de este caso de uso puede consultarse en la Figura A.5 del Anexo A.*

CU15 - Abrir el Dashboard

Código único: CU15

Nombre: Abrir Dashboard

Precondición: El usuario está autenticado en la aplicación.

Escenario principal:

1. El actor accede a la sección principal de la aplicación.
2. El sistema consulta a la BBDD todo el historial vinculado al ID del usuario.
3. El sistema calcula dinámicamente las sumas de protocolos seguros, vulnerables y el desglose de ataques.
4. El sistema renderiza los contadores, gráficos y la lista de protocolos recientes en la interfaz visual.

Escenario alternativo:

- **2a.** La base de datos del usuario está en blanco (usuario recién registrado).
- **2b.** El sistema carga el Dashboard mostrando todas las métricas a cero y la lista vacía.

Postcondición: El panel de control queda cargado en pantalla con la información de rendimiento actualizada.

CU16 - Eliminar protocolo

Código único: CU16

Nombre: Eliminar protocolo

Precondición: El usuario está en el Dashboard y tiene protocolos guardados en la lista de recientes.

Escenario principal:

1. El actor selecciona un protocolo concreto del historial y pulsa eliminar.
2. El sistema muestra una alerta de confirmación.
3. El actor confirma el borrado.

4. El sistema localiza el ID del protocolo en la BBDD y elimina su registro junto a sus resultados.
5. El sistema recalcula las métricas estadísticas para reflejar el borrado.
6. El sistema refresca visualmente la lista del Dashboard.

Escenario alternativo:

- **3a.** El actor rechaza la eliminación en la alerta.
- **3b.** El sistema cierra la alerta sin realizar cambios en la BBDD.

Postcondición: El protocolo seleccionado y sus estadísticas asociadas desaparecen permanentemente.

CU17 - Cargar protocolo

Código único: CU17

Nombre: Cargar protocolo

Precondición: El usuario está en el Dashboard y tiene protocolos guardados en su historial.

Escenario principal:

1. El actor hace doble clic en uno de los elementos de la lista de protocolos recientes o pulsa en el botón de acceder.
2. El sistema consulta a la BBDD la ruta (path) del archivo asociado a ese protocolo.
3. El sistema localiza y abre el archivo físico en el disco duro.
4. El sistema transiciona a la pantalla del editor de texto y vuelca la información, dejándolo listo para su edición.

Escenario alternativo:

- **3a.** El archivo referenciado ya no existe en el disco duro o ha sido movido externamente por el usuario.
- **3b.** El sistema muestra un mensaje de advertencia y ofrece la opción de eliminar el registro huérfano de la BBDD, manteniéndose en el Dashboard.

Postcondición: El modelo cargado desde el disco duro pasa a ser el modelo activo de trabajo.

CU18 – Renombrar protocolo

Código único: CU18

Nombre: Renombrar protocolo

Precondición: El usuario está en el Dashboard y tiene protocolos guardados.

Escenario principal:

1. El actor pulsa el icono de edición sobre el nombre de un protocolo del historial.
2. El sistema habilita la escritura sobre esa celda/campo.
3. El actor teclea el nuevo nombre identificativo y pulsa la tecla intro para confirmar.
4. El sistema ejecuta una sentencia UPDATE en la BBDD vinculando el nuevo nombre a ese ID.
5. El sistema actualiza el nombre visible en la lista.

Escenario alternativo:

- **3a.** El actor deja el nombre completamente en blanco y confirma.
- **3b.** El sistema lo detecta como no válido, deshace el cambio y restaura el nombre anterior.

Postcondición: El alias de identificación del protocolo se modifica en el sistema.

4

Diseño y Arquitectura

4.1 Arquitectura General

Se ha diseñado la arquitectura de este *software* usando el patrón arquitectónico Modelo-Vista-Controlador (MVC) [9], creando 4 capas diferentes para estructurarlo. El haber tomado esta decisión sobre el diseño garantiza que la aplicación presente un alto nivel de desacoplamiento, por lo que se facilita la escalabilidad y su mantenimiento a largo plazo.

Las capas que componen el sistema son las siguientes:

- **Capa de Presentación (Vista):** Implementada mediante el *framework* JavaFX. En esta capa encontramos los controladores (`ControladorLogin`, `ControladorAnalisis`, `ControladorPrincipal`, `ControladorPerfil`, etc.) y las interfaces gráficas con las que interactúa el usuario. Su responsabilidad se limita a la captura de eventos y representación visual, pero no llega a realizar ningún cálculo criptográfico ni interacción fuera de su función.
- **Capa Lógica (Controlador/Motor/Servicios):** Conformar el núcleo operativo de la aplicación. Aquí se agrupan componentes esenciales tales como el `Analizador Léxico`, el `Motor de análisis` y el `Gestor de Sesión`. Adicionalmente, se ha implementado una subcapa de servicios (`ServicioDashboard`, `ServicioPerfil`) que hace las veces de intermediaria entre los controladores de la vista y la capa de persistencia, garantizando el Modelo-Vista-Controlador. Además, es la capa encargada de procesar las reglas del negocio y validar la sintaxis de las entradas, a la vez que maneja la detección de vulnerabilidades.
- **Capa de Persistencia:** Encargada de la gestión de información y de evitar que se produzcan irregularidades en esta. Incluye el `Gestor de BBDD` encargado de la comunicación directa con

SQLite, y el **Gestor de archivos**, responsable de lo relacionado con el manejo de archivos, como la importación/exportación o la generación de informes en formato PDF.

- **Capa de Utilidades Transversales (util):** Paquete de soporte que actúa de forma transversal al sistema. Centraliza la navegación y el control del flujo de pantallas a través del **GestorVistas**, abstrae la generación de alertas visuales mediante el **GestorDialogos**, y maneja la comunicación de red para correos electrónicos con el **ServicioEmail**.
- **Capa de Modelo:** En ella se definen las entidades que conforman el dominio de la aplicación. Contiene todo lo relacionado con el modelo de datos, desde los registros persistentes (**Usuario**, **Protocolo**, **Vulnerabilidad**) hasta la estructura de datos que conforma el Árbol Sintáctico Abstracto (**Mensaje**, **Agente**, **Nonce**, etc.).

Encapsulamiento Defensivo del Modelo: Para prevenir mutaciones externas accidentales sobre las estructuras internas del AST, todos los métodos de acceso (*getters*) que devuelven colecciones en las clases del modelo (**Protocolo**, **Mensaje**, **Cifrado**, **Clave**) retornan vistas inmutables mediante `Collections.unmodifiableList()`. Intentar modificar cualquiera de estas listas desde fuera lanzará una `UnsupportedOperationException`.

Esta separación arquitectónica estricta se fundamenta en las relaciones de dependencia y asociación direccional que comunican los diferentes módulos del sistema. A continuación, en las Figuras 4.1 y 4.2, se detallan los paquetes centrales del diseño correspondientes a las capas de Lógica y Modelo.

Para un análisis exhaustivo de la arquitectura, tanto el diagrama de clases global del sistema como la división de este en paquetes pueden consultarse en el **Anexo B**.

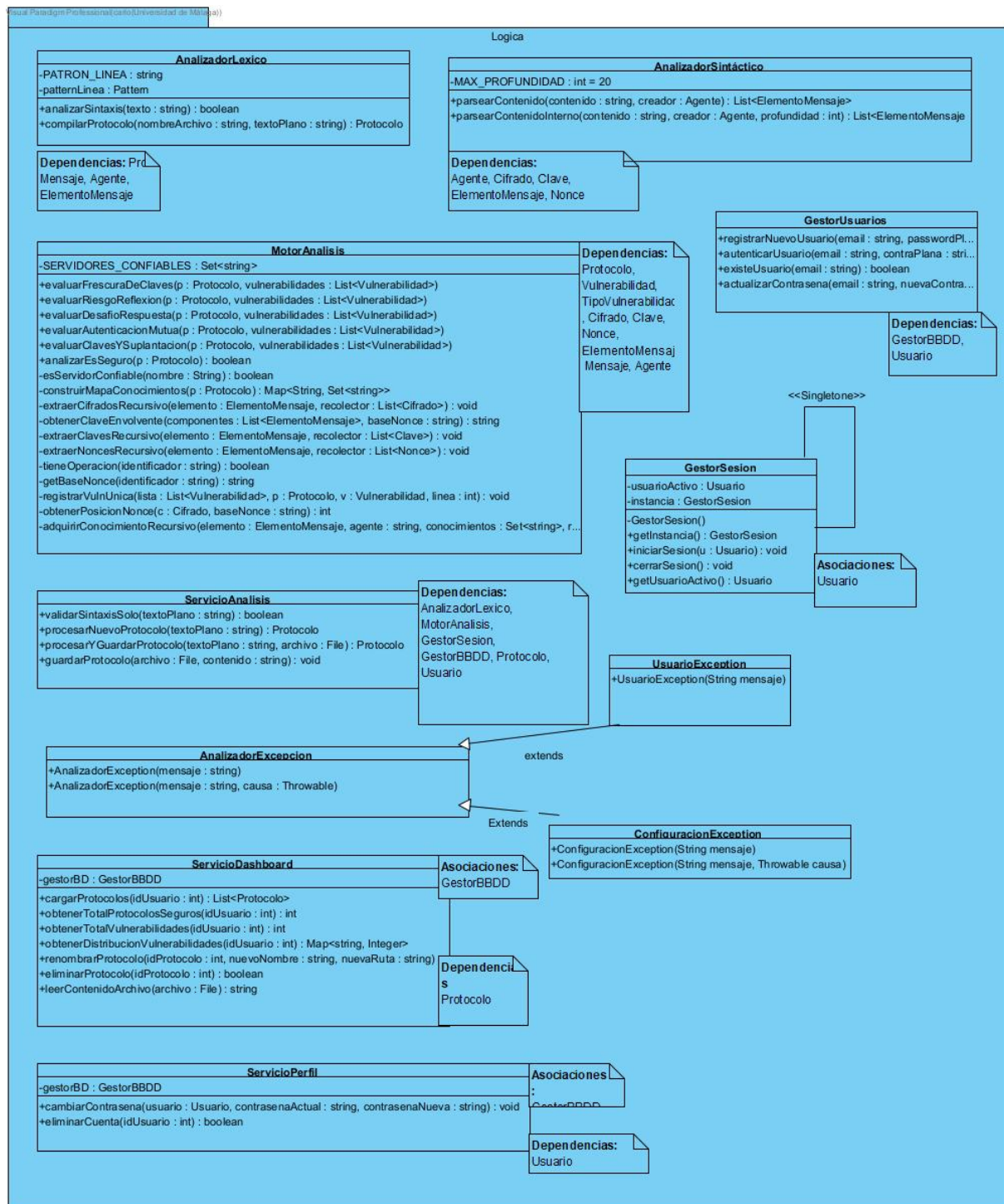


Figura 4.1. Paquete de Lógica del diagrama de clases de implementación y arquitectura por capas.

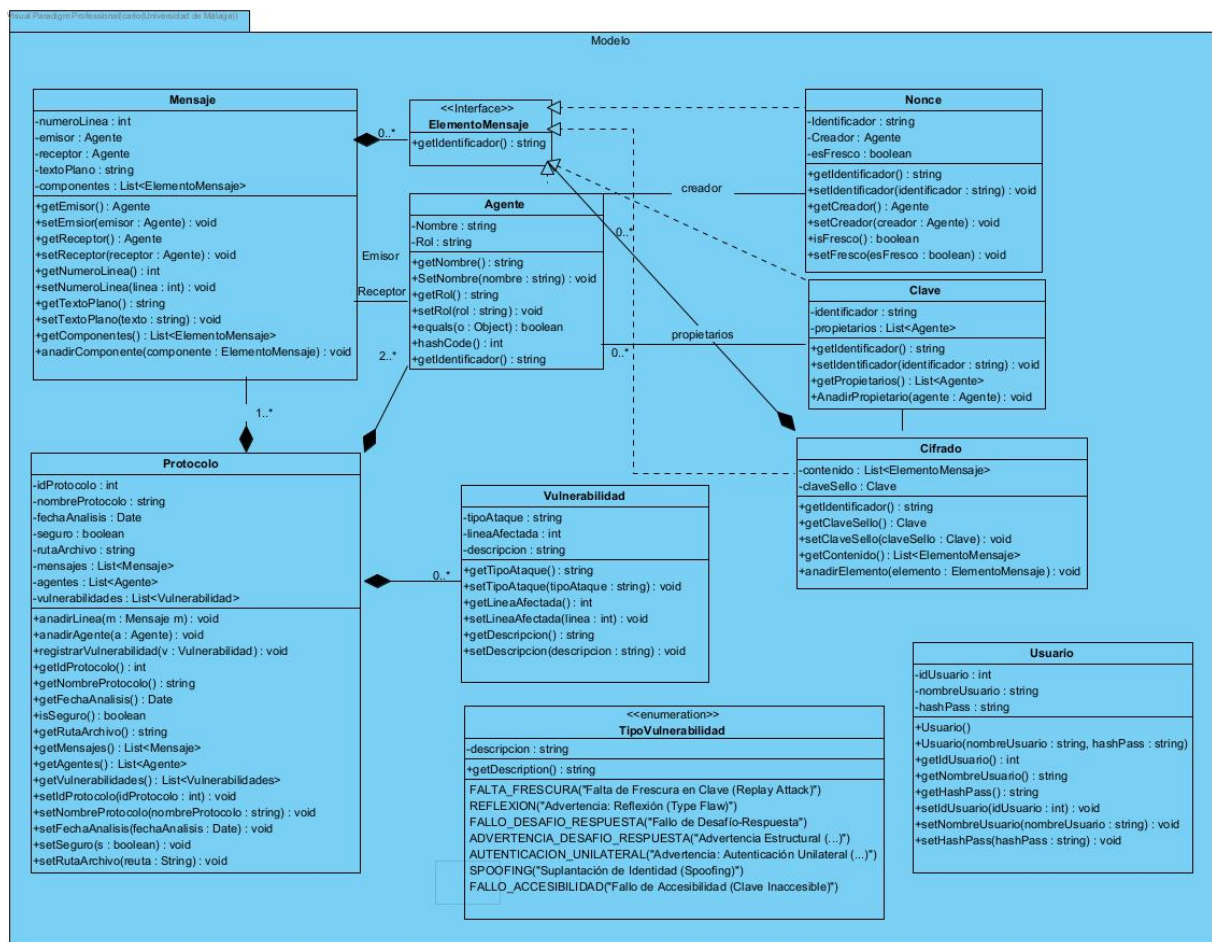


Figura 4.2. Paquete de Modelo del diagrama de clases de implementación y arquitectura por capas.

4.1.1 Jerarquía de Excepciones del Dominio

Con el objetivo de mejorar la captura de errores y sustituir el uso de excepciones genéricas de Java, se ha diseñado una jerarquía de excepciones propias:

- **AnalizadorException:** Excepción base del proyecto, lanzada ante errores internos del motor de análisis.
- **UsuarioException:** Usada para errores relacionados con la autenticación y gestión de cuentas.
- **ConfiguracionException:** Usada para errores de carga del archivo `config.properties` o ausencia de credenciales SMTP.

Esta decisión de diseño sigue el principio de *Fail Fast*, haciendo que cada capa del sistema capture y gestione solo las excepciones que le competen.

4.2 Diseño de la Lógica (Motor)

El motor de análisis conforma el núcleo de la aplicación y es el encargado de procesar y evaluar las sentencias introducidas por el usuario. Para lograrlo, un analizador de descenso recursivo transforma el texto plano en una estructura de datos jerárquica en memoria: el Árbol Sintáctico Abstracto (AST).

4.2.1 Patrón Estructural: Composite

La notación criptográfica es inherentemente recursiva. Un mensaje puede contener varios elementos como texto plano, *nonces*, claves o bloques cifrados. A su vez, estos bloques cifrados pueden incluir en su interior otros bloques cifrados, creando así varios niveles de anidamiento criptográfico.

Se ha implementado el patrón de diseño estructural *Composite* [3] con el objetivo de modelar esta recursividad sin saturar la lógica del motor con múltiples sentencias condicionales. Este patrón permite que tanto objetos individuales como composiciones de estos sean tratados uniformemente.

La arquitectura del AST se sustenta en las siguientes piezas fundamentales:

- **Componente (ElementoMensaje):** Interfaz base que define el contrato común para todos los nodos del árbol. Exige la implementación del método `getIdentificador()`.
- **Hojas o Leaves (Agente, Nonce, Clave):** Estas clases representan los elementos criptográficos de nivel atómico (nodos hoja). Actúan como los nodos finales del AST, pues estos no pueden contener elementos.
- **Compuesto o Composite (Cifrado):** Clase que representa un bloque de información protegido por una clave simétrica. Al implementar la interfaz `ElementoMensaje` y poseer al mismo tiempo una colección interna de dichos elementos (`List<ElementoMensaje> contenido`), permite encapsular de forma recursiva cualquier combinación de hojas u otros cifrados.

Finalmente, la clase `Mensaje` actúa como la raíz (contexto) para cada línea evaluada, almacenando la secuencia completa de componentes y los metadatos de los agentes emisor y receptor. La orquestación y creación de estos nodos recae sobre la clase `AnalizadorSintactico`, que resuelve la anidación delegando en sus propios métodos de parseo.

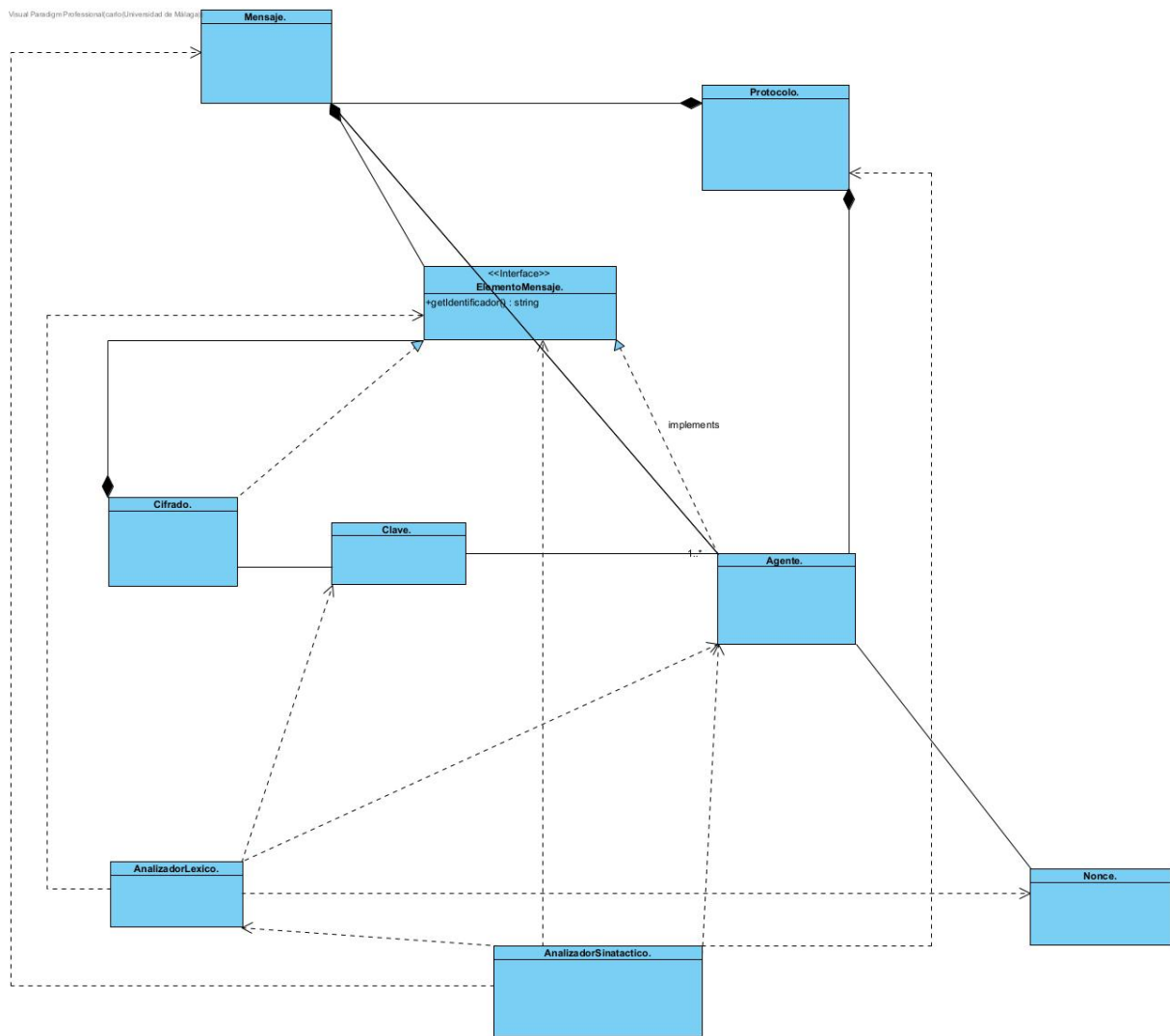


Figura 4.3. Diagrama de clases del Analizador Sintáctico y la estructura del AST (Patrón Composite) simplificado (versión completa en el anexo).

4.3 Diseño de Datos

Con el objetivo de garantizar que se mantenga la persistencia de la información a la vez que la ligereza y portabilidad de la herramienta, se ha empleado un motor de base de datos relacional embebido mediante SQLite. Con el fin de evitar redundancias, se ha normalizado el modelo de datos, el cual presenta tres entidades principales conectadas entre sí.

4.3.1 Diccionario de Datos

4.3.1.1 Entidad: USUARIO

Tabla encargada de almacenar las credenciales de acceso de los usuarios. Cuenta con una restricción de tipo *Unique* estricta aplicada sobre el correo electrónico para evitar cuentas duplicadas y permitir la correcta verificación mediante el protocolo SMTP.

Columna	Tipo de Dato	Restricciones
id_usuario	INTEGER	PRIMARY KEY, AUTOINCREMENT
correo_electronico	TEXT	UNIQUE, NOT NULL
hash_contra	TEXT	NOT NULL

Tabla 4.1. Esquema de la tabla USUARIO.

4.3.1.2 Entidad: PROTOCOLO

Tabla encargada de gestionar el historial de las evaluaciones realizadas por cada usuario. Se ha escogido el guardar únicamente el puntero físico hacia el archivo en el disco duro en lugar de almacenar el texto del propio protocolo para así evitar sobrecargar el sistema.

Columna	Tipo de Dato	Restricciones
id_protocolo	INTEGER	PRIMARY KEY, AUTOINCREMENT
id_usuario	INTEGER	FOREIGN KEY → USUARIO (ON DELETE CASCADE)
nombre_protocolo	TEXT	NOT NULL
fecha_analisis	TEXT	NULLABLE (Formato ISO8601)
es_seguro	INTEGER	NULLABLE (Booleano: 0 o 1)
ruta_archivo	TEXT	NOT NULL

Tabla 4.2. Esquema de la tabla PROTOCOLO.

4.3.1.3 Entidad: VULNERABILIDAD

Tabla encargada de llevar un registro detallado de cada una de las brechas de seguridad que el motor lógico ha detectado durante el análisis del protocolo objetivo.

Columna	Tipo de Dato	Restricciones
id_vulnerabilidad	INTEGER	PRIMARY KEY, AUTOINCREMENT
id_protocolo	INTEGER	FOREIGN KEY → PROTOCOLO (ON DELETE CASCADE)
nombre	TEXT	NOT NULL
tipo	TEXT	NOT NULL
linea	INTEGER	NOT NULL
descripcion	TEXT	NULLABLE

Tabla 4.3. Esquema de la tabla VULNERABILIDAD.

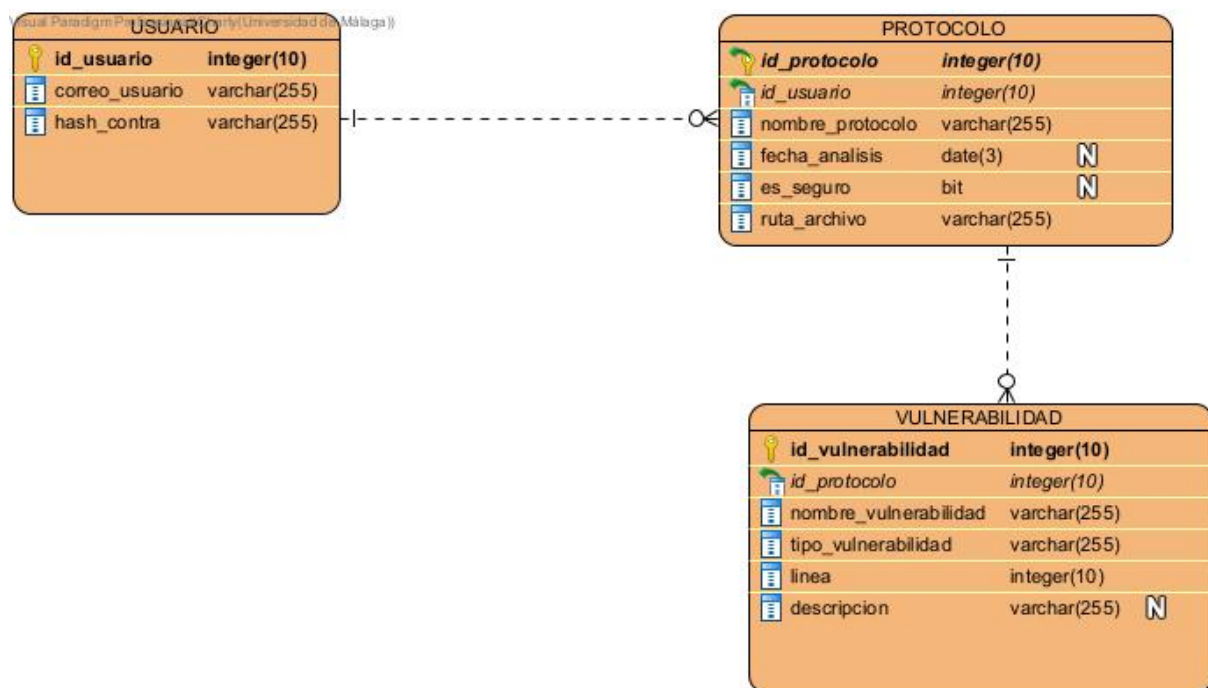


Figura 4.4. Modelo Entidad-Relación de la base de datos local SQLite.

4.4 Diseño de Interfaz

El diseño de la interfaz gráfica de usuario (GUI) se ha llevado a cabo estableciendo unos principios estrictos de usabilidad, garantizando que el usuario (ya sea investigador o estudiante) pueda acceder a cualquier funcionalidad esencial de análisis en un máximo de cuatro clics (requisito RNF3). Debido al uso del patrón Modelo-Vista-Controlador (MVC), la capa visual, cuya implementación será realizada mediante el uso de JavaFX, elimina totalmente el acoplamiento con respecto a la lógica criptográfica

subyacente.

A continuación, se presentan los *mockups* (bocetos estructurales) que definen la distribución de los distintos módulos de la aplicación.

4.4.1 Acceso al Sistema

La toma de contacto del usuario con el sistema se da a través de las interfaces de control de acceso. Contando con un diseño minimalista, se centran en los formularios de autenticación y registro de nuevas credenciales, en este punto se incluye la verificación por correo electrónico.

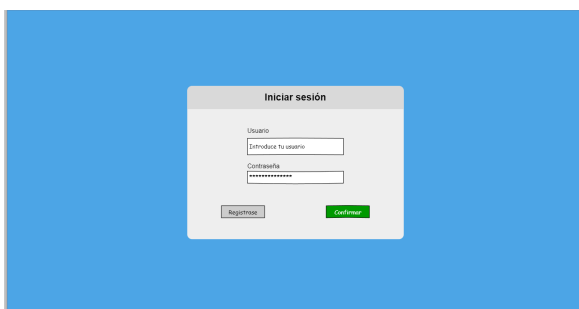


Figura 4.5. *Interfaz de Inicio de Sesión.*

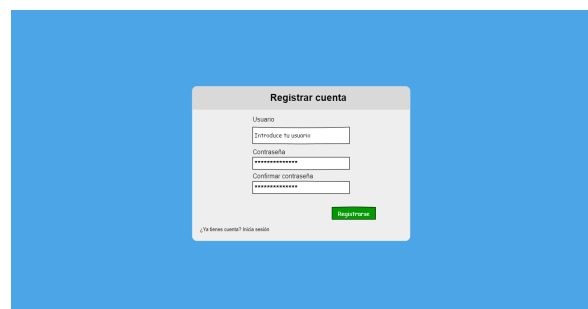


Figura 4.6. *Interfaz de Registro de Cuenta.*

4.4.2 Panel de Control (Dashboard)

Tras la autenticación, se lleva al usuario a la vista principal de la herramienta. En esta pantalla, que actúa como centro de mando, se divide el espacio en dos áreas, una donde se mostrarán gráficas de las métricas de vulnerabilidades registradas y otra área operativa, que permite la gestión rápida del historial de protocolos (consultar, editar nombre o eliminar).

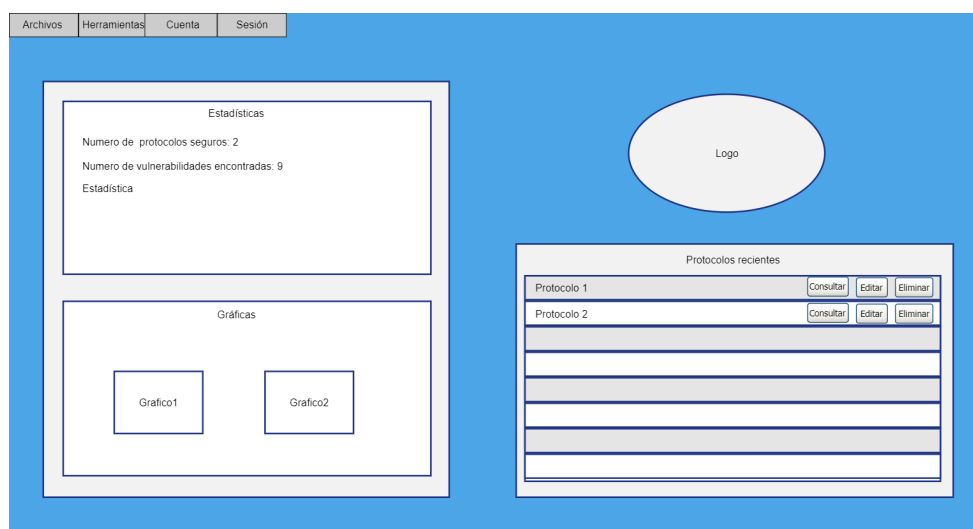


Figura 4.7. Vista principal del panel de control (Dashboard).

4.4.3 Entorno de Especificación y Análisis

El núcleo del funcionamiento del *software* es el editor de protocolos. Esta interfaz se divide en tres partes lateralmente: a la izquierda se encuentran las herramientas de configuración y validación; en el centro un cuadro de texto enumerado donde se introducirá el protocolo; y a la derecha, encontramos el panel que muestra los resultados del análisis y el botón encargado de crear el informe del protocolo analizado.

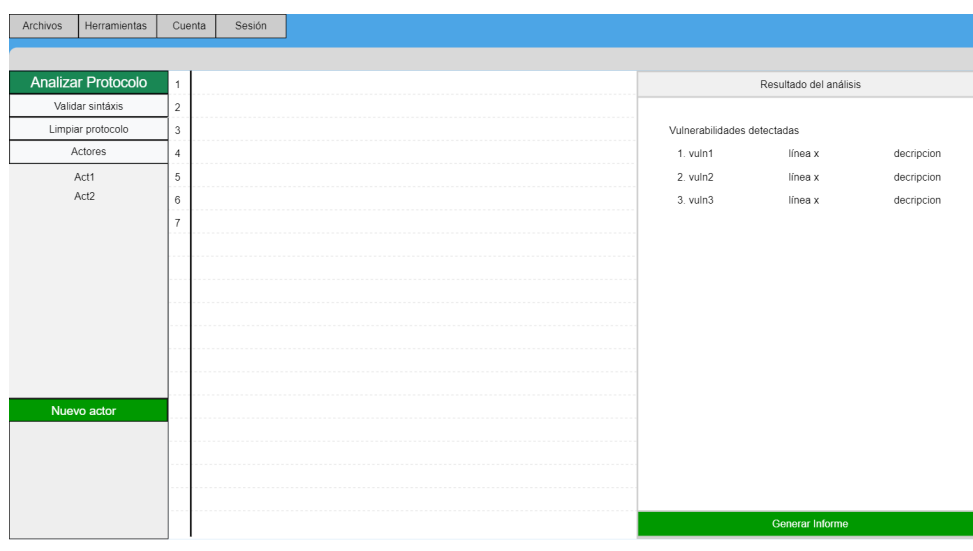


Figura 4.8. Entorno del editor de texto y volcado de resultados analíticos.

4.4.4 Gestión de Usuario y Soporte Integrado

Se han diseñado vistas específicas para la configuración de la cuenta y para la ayuda al usuario, para así evitar depender de fuentes externas para ello. La gestión de la cuenta permite el cambio de contraseñas de forma segura, mientras que la guía integrada (RNF4) proporciona la documentación estática necesaria sobre la sintaxis soportada por el compilador y cómo ejecutar el análisis correctamente sin necesidad de abandonar la aplicación.

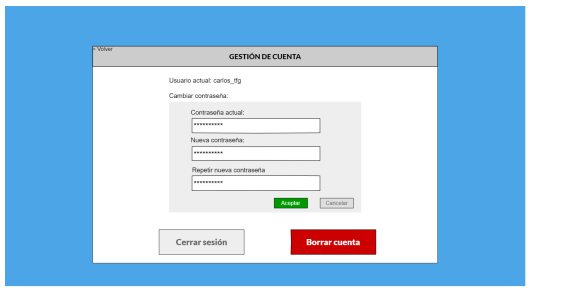


Figura 4.9. Interfaz de gestión de cuenta.

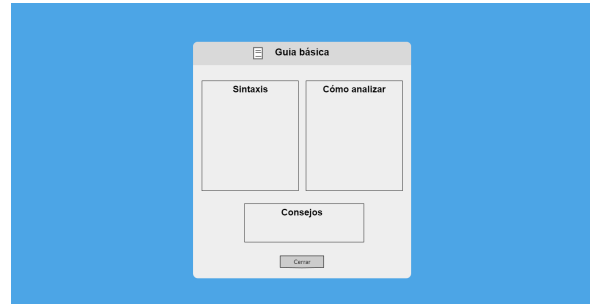


Figura 4.10. Guía estática de sintaxis y uso.

4.4.5 Seguridad en Acciones Irreversibles

Para evitar problemas con la integridad de los datos en la base de datos local, han sido diseñadas ventanas emergentes (*modals*) con el fin de exigir una confirmación explícita del usuario antes de realizar acciones irreversibles como ejecutar sentencias de borrado (DELETE), evitando así pérdida accidental de datos.



Figura 4.11. Confirmación de borrado de protocolo.



Figura 4.12. Confirmación de eliminación de cuenta.

5

Implementación

5.1 Tecnologías Utilizadas

Para el desarrollo de la herramienta ha sido requerida la integración de un conjunto de tecnologías y librerías seleccionadas estratégicamente para cumplir tanto con las especificaciones funcionales como con las restricciones de rendimiento, robustez y seguridad (requisitos RNF). La elección de este *stack* tecnológico garantiza la portabilidad de la aplicación de escritorio y el respeto al patrón de diseño arquitectónico implementado.

A continuación, se justifican técnicamente los componentes tecnológicos principales del sistema:

5.1.1 Lenguaje de Programación: Java

Para el desarrollo del sistema se ha seleccionado Java como lenguaje principal, empleando la especificación de la versión de soporte a largo plazo Java 25 LTS. La elección se fundamenta en su robustez, su tipado fuerte y estático, y su naturaleza orientada a objetos, cualidades necesarias para la correcta modelación de las estructuras jerárquicas complejas del Árbol Sintáctico Abstracto (AST) mediante el patrón *Composite*.

El uso de esta versión de vanguardia permite beneficiarse de las últimas optimizaciones en la Máquina Virtual de Java (JVM) y en el recolector de basura (*Garbage Collector*), mitigando el riesgo de fugas de memoria durante el análisis recursivo de protocolos extensos. Por último, la independencia de la plataforma que ofrece la JVM asegura el cumplimiento de la portabilidad de la herramienta entre diferentes sistemas operativos.

5.1.2 Interfaz Gráfica de Usuario: JavaFX

Para la construcción de la interfaz gráfica e interactiva (GUI) se ha empleado *JavaFX* [5]. A diferencia de entornos más antiguos como *Swing*, *JavaFX* proporciona un soporte nativo y optimizado para la implementación estricta del patrón Modelo-Vista-Controlador (MVC). Además, es compatible con herramientas de diseño visual como *Scene Builder*, lo que agiliza significativamente la maquetación de los componentes sin interferir en el código fuente.

La separación de responsabilidades se materializa mediante el uso de archivos estructurados en formato FXML para la definición declarativa de las vistas, vinculados de forma unívoca a sus respectivas clases controladoras en Java mediante anotaciones `@FXML`. Adicionalmente, el motor gráfico de *JavaFX* gestiona de forma eficiente la concurrencia en la interfaz, permitiendo delegar tareas pesadas (como el envío de correos SMTP) a hilos de trabajo secundarios (*Background Threads*), evitando así que se congele o comprometa la respuesta del hilo principal de la aplicación.

Para mejorar la experiencia de usuario en dispositivos con diferentes resoluciones de pantalla, la ventana principal de la aplicación ha sido configurada como redimensionable, con unas dimensiones mínimas de 1024×700 píxeles con el fin de evitar que se lleguen a deformar los componentes de la interfaz. Adicionalmente, el gestor de vistas (*GestorVistas*) preserva las dimensiones y el estado de maximización de la ventana durante las transiciones entre pantallas, evitando el reinicio del tamaño al cambiar de escena. En aquellas vistas cuyo contenido puede exceder el espacio vertical disponible, como la gestión de cuenta, se ha integrado un componente *ScrollPane* que permite el desplazamiento del contenido sin que se pierda ninguna funcionalidad.

5.1.3 Sistema de Persistencia: SQLite

La persistencia de datos local, referente al control de accesos, métricas y paths del historial de protocolos, se ha resuelto usando el motor de base de datos relacional *SQLite* [4]. Al ser una base de datos embebida, no requiere la instalación ni configuración de un servidor de base de datos independiente en el equipo del usuario; la información se almacena en un archivo físico local (*analisis.db*).

La comunicación entre la capa de servicios lógicos de Java y la base de datos se realiza a través de la API estándar JDBC (*Java Database Connectivity*), garantizando la integridad referencial y transaccional del modelo entidad-relación mediante consultas parametrizadas que previenen cualquier vulnerabilidad de inyección SQL. Además, con el fin de evitar que los datos se corrompan debido a fallos parciales durante el procesamiento, se ejecutan operaciones compuestas de actualización del historial

dentro de transacciones atómicas con mecanismo de *rollback* automático.

5.1.4 Librerías y Dependencias Auxiliares

Para complementar las capacidades nativas del lenguaje y resolver requerimientos específicos de seguridad y generación de documentos externos, se han integrado las siguientes librerías de código abierto:

- **jBCrypt:** Utilizada en la capa de lógica de usuarios y servicios (`GestorUsuarios` y `ServicioPerfil`) para la protección de las credenciales de acceso locales. Implementa el algoritmo de derivación de claves adaptativo basado en el cifrado Blowfish, aplicando un *salt* aleatorio único por cada registro. Esto garantiza el cumplimiento del requisito RNF5, protegiendo las contraseñas contra ataques de tablas de búsqueda (*Rainbow Tables*) y ataques de fuerza bruta en el almacenamiento local.
- **iText PDF:** Incorporada en el `GestorArchivos` para resolver el requisito funcional de generación de informes técnicos estructurados (RF9). Esta librería permite maquetar y exportar dinámicamente en formato PDF los resultados analíticos guardados en memoria, traduciendo los listados de la clase `Protocolo` y sus correspondientes objetos `Vulnerabilidad` a un documento formal legible externamente.
- **RichTextFX (v0.11.3):** Es una librería de código abierto integrada cuya función es proporcionar el componente `CodeArea` del editor de protocolos. A diferencia del `TextArea` estándar de *JavaFX*, `CodeArea` permite la numeración de líneas nativa (requisito RF3.3) y mantiene la posibilidad de llevar a cabo la futura implementación del resaltado de sintaxis asíncrono descrito en el trabajo futuro del proyecto, sin interferir en el hilo principal de la aplicación.

5.1.5 Gestión de Credenciales SMTP y Modelado de Amenazas

Es necesario almacenar una credencial de autenticación en el dispositivo del usuario para que la comunicación con el servidor SMTP de Google funcione correctamente. Aunque se evaluó la posibilidad de cifrar el archivo localmente, esta alternativa fue descartada debido a que la clave de descifrado debería residir en el propio código fuente compilado, siendo trivialmente extraíble mediante un descompilador de *bytecode* Java.

En lugar de esto último, se ha aplicado un enfoque siguiendo el **Principio de Mínimo Privilegio** y la **Reducción de Impacto**:

- **Mínimo Privilegio:** La credencial empleada es una *Contraseña de Aplicación* generada desde Google, únicamente utilizable para los protocolos SMTP/IMAP. Esta credencial no permite acceder a la cuenta de Google del usuario (no se puede iniciar sesión web, acceder a Google Drive, ni modificar la configuración de la cuenta).
- **Reducción de Impacto (*Blast Radius*):** La cuenta de correo utilizada se ha creado únicamente con el fin de ser usada para el proyecto, por lo que no contiene datos personales ni contactos vinculados. En caso de que un atacante obtuviese acceso al archivo de configuración local, el impacto sobre la privacidad del usuario sería nulo.
- **Exclusión del repositorio:** El archivo `config.properties` real está incluido en el `.gitignore` del proyecto, evitando que se llegue a publicar en el repositorio remoto. En su lugar, se proporciona un archivo de ejemplo (`config.properties.example`) con instrucciones de configuración.

5.1.6 Herramientas de Desarrollo y Gestión

Además del *stack* tecnológico principal, el desarrollo del proyecto ha requerido el uso de las siguientes herramientas de soporte:

- **Apache Maven:** Sistema de gestión y construcción del proyecto (*build tool*). Gestiona de forma declarativa todas las dependencias listadas en el archivo `pom.xml`, automatiza la compilación y orquesta la ejecución de la suite de pruebas unitarias mediante el plugin *Surefire* con el comando `./mvnw.cmd clean test`.
- **GitHub y GitHub Desktop:** Sistema de control de versiones distribuido empleado para garantizar la integridad del código fuente y llevar un seguimiento ordenado de la evolución del proyecto. El uso de *GitHub Desktop* ha simplificado la gestión del flujo de trabajo sin necesidad de emplear la línea de comandos como (`commit`) y (`push/pull`).
- **Scene Builder:** Herramienta de diseño visual de interfaces de *JavaFX* empleada para maquetar los archivos `.fxml` de cada vista de la aplicación. Permite la creación gráfica de los componentes de la interfaz (botones, paneles, campos de texto) sin escribir el código FXML.

manualmente, reduciendo el tiempo de diseño visual y previniendo errores estructurales en las vistas.

- **Visual Paradigm:** Herramienta CASE (*Computer-Aided Software Engineering*) utilizada para la elaboración de la mayoría de los diagramas UML del proyecto: diagrama de clases por capas, diagrama de casos de uso y diagrama Entidad-Relación.
- **Trello:** Herramienta de gestión de tareas basada en la metodología *Kanban*, empleada para la planificación y seguimiento del desarrollo por *sprints*. La organización en listas (*Backlog*, *Pendientes*, *En Progreso* y *Hecho*) ha permitido mantener una visión clara del estado de cada funcionalidad en las reuniones periódicas con el tutor.

5.2 Desarrollo del Analizador

Para llevar a cabo el procesamiento criptográfico el *software* cuenta con dos módulos coordinados. El primero de ellos, el analizador léxico, se ha implementado empleando Java sin usar dependencias externas para así garantizar la mayor eficiencia computacional posible y evitar el acoplamiento a librerías de terceros.

5.2.1 Analizador Léxico y Validación Estructural

La primera fase es el análisis léxico, proceso en el cual se toma una sentencia bruta (mensaje) y se divide en unidades lógicas individuales con sentido para el sistema (emisor, receptor y contenido). Para llevar a cabo esta tarea y la validación de la gramática estándar de protocolos, se ha hecho uso de la librería estándar `java.util.regex`. El núcleo de este módulo reside en la compilación estática de una Expresión Regular (Regex) que actúa como un autómata finito capaz de reconocer la estructura base de cualquier comunicación:

```
1 private static final String PATRON_LINEA = "^\\s*([a-zA-Z0-9_]+)\\s*->\\s*\n    *([a-zA-Z0-9_]+)\\s*:\\s*(.+)$";
```

Listing 5.1. Patrón de validación estructural de mensajes.

Este patrón se encarga de segmentar cada línea del editor en tres grupos de unidades lógicas:

1. **Grupo 1 (Emisor):** Cadena alfanumérica previa al operador direccional `->`.

2. **Grupo 2 (Receptor):** Cadena alfanumérica posterior a la flecha y previa a los dos puntos :.
3. **Grupo 3 (Contenido):** El resto de la cadena de texto, que incluye el contenido con significado del mensaje (texto plano, claves o elementos cifrados).

En el proceso de lectura del texto plano, la clase `AnalizadorLexico` se encarga de iterar secuencialmente aplicando el patrón de búsqueda dado por la expresión regular. En el momento en el que el sistema reconoce que una línea no coincide con este patrón de forma estricta, se interrumpe la ejecución a la vez que se lanza una excepción controlada de tipo `AnalizadorException`. Este mecanismo garantiza el cumplimiento del requisito de robustez (RNF2), notificando al usuario del error sintáctico sin provocar el cierre inesperado de la aplicación.

En caso de que la validación sea exitosa, el sistema toma los grupos capturados, instancia los objetos de la clase `Agente` (emisor y receptor) valiéndose de la sobrescritura del método `equals()`, y encapsula el resto del contenido presente dentro de un objeto `Mensaje`. Estas instancias son agregadas secuencialmente a la estructura de la clase `Protocolo`, dejándola lista para la segunda fase del procesamiento.

Una vez superada la fase léxica, el `AnalizadorLexico` no construye el Árbol Sintáctico Abstracto (AST) por sí mismo, sino que delega el contenido bruto en la clase `AnalizadorSintactico` invocando su método `parsearContenido()`. Este analizador actúa como el verdadero constructor del AST, resolviendo de forma recursiva los bloques del patrón *Composite* [3]. Además, las instancias resultantes (como `Nonce` o `Clave`) no se generan de manera aislada; la implementación realiza un seguimiento estricto de sus creadores y propietarios legítimos, inyectando esta información en el modelo para alimentar posteriormente las reglas de auditoría del motor criptográfico.

5.3 Desarrollo del Motor Criptográfico

Tras haber realizado las fases léxica y sintáctica, el Árbol Sintáctico Abstracto (AST) ya estará totalmente estructurado en la memoria del sistema. La clase `MotorAnálisis`, que conforma el núcleo analítico del *software*, es la responsable de auditar este modelo. La base teórica sobre la que se fundamenta el diseño de este motor corresponde a las premisas del modelo del intruso de Dolev-Yao, donde se asume que la criptografía es matemáticamente perfecta, pero que existe un atacante que tiene el control total para poder interceptar, bloquear, inyectar o alterar el orden de los mensajes.

Como en la estructura del AST se hace uso del llamado patrón *Composite*, el motor, en lugar

de seguir una estructura lineal de lectura, implementa una serie de algoritmos de búsqueda y evaluación basados en el recorrido recursivo de los nodos del AST. Métodos internos como pueden ser `extraerNoncesRecursivo()` o `adquirirConocimientoRecursivo()` se encargan de realizar el descenso a través de los diferentes niveles de anidamiento criptográfico que se pueden llegar a observar en la clase `Cifrado`, lo que permite clasificar la información y claves que posee cada agente en todo momento.

El funcionamiento de la clase `MotorAnálisis` consiste en someter al protocolo a una batería de pruebas lógicas secuenciales, en las que evalúa de forma independiente las diferentes propiedades de seguridad requeridas para validar la robustez del diseño, cuya ausencia desencadenaría vulnerabilidades. En los siguientes párrafos serán detallados los módulos de evaluación que implementa:

5.3.1 Soporte de Operaciones Funcionales sobre Nonces

Además de las operaciones clásicas sobre los *nonces*, como $N_A - 1$ o $N_A + 1$, se ha dotado al analizador sintáctico de la capacidad de reconocer el uso de funciones criptográficas sobre estos elementos. Así, puede reconocer expresiones como $F(N_A)$, $\text{Hash}(N_{\text{Bob}})$ o $\text{SHA256}(N_B)$ como *nonces* mutados en lugar de verlos como agentes.

Esta extensión afecta a tres puntos del sistema:

- **Analizador Sintáctico:** Añadida una regla de detección previa a la clasificación de *nonces*: si un *token* contiene paréntesis y en su interior se encuentra un *nonce* o marca de tiempo, se clasifica como *Nonce*. El sistema exige la existencia de un nombre de función antes del paréntesis de apertura, y en caso de no existir se lanza un `AnalizadorException` indicando el error de sintaxis.
- **Detección de mutación (`tieneOperacion`):** El método auxiliar clasifica ahora los paréntesis `()` como indicadores de operación, manteniendo además los operadores `+` y `-`.
- **Extracción de base (`getBaseNonce`):** El método extrae el contenido de los paréntesis $F(N_A) \rightarrow N_A$ antes de aplicar la limpieza aritmética clásica.

Esta mejora permite que el motor trate la aplicación de una función criptográfica como una transformación legítima que rompe la simetría de reflexión, alineándose con las prácticas estándar de los protocolos de desafío-respuesta modernos donde se emplea $F(N)$ en lugar de solo $N - 1$.

5.3.2 Centralización de Tipos de Vulnerabilidad

Para evitar la acumulación de las mismas cadenas de texto literales (*magic strings*) a lo largo de las reglas del motor, se ha creado el *enum* `TipoVulnerabilidad`, que centraliza los diferentes tipos de fallo detectables: `FALTA_FRESCURA`, `REFLEXION` (la cual es una advertencia), `FALLO_DESAFIO_RESPUESTA`, `ADVERTENCIA_DESAFIO_RESPUESTA`, `AUTENTICACION_UNILATERAL`, `SPOOFING` y `FALLO_ACCESIBILIDAD`. Cada constante encapsula su descripción legible, garantizando la consistencia del diagnóstico y facilitando la posibilidad de extender el motor a futuro.

5.3.3 Evaluación de Frescura y Prevención de *Replay Attacks*

Asegurar la frescura (*freshness*) es necesario para garantizar que los elementos presentes en la sesión (como los *tickets* o claves temporales) realmente pertenezcan a la comunicación que se lleva a cabo en este momento, evitando que sean paquetes antiguos reinyectados por el atacante.

El método `evaluarFrescuraDeClaves()` permite al motor recorrer todos los mensajes mientras busca la distribución de claves de sesión, que normalmente son emitidas por un servidor KDC. Esta distribución se considera segura cuando la clave distribuida viaja encapsulada junto a un *Nonce* o a una marca de tiempo emitida previamente por el receptor de ese mensaje. Si estas garantías no están presentes durante el envío de claves o *tickets* opacos, el sistema indica que existe una vulnerabilidad de tipo "Falta de Frescura", la registra y alerta de que el protocolo es vulnerable ante ataques de repetición (*Replay Attacks*).

5.3.4 Mecanismos de Desafío-Respuesta

El Desafío-Respuesta es empleado por los protocolos modernos para así asegurar la identidad de los interlocutores y evaluar la vitalidad de la conexión (*liveness*). En el motor, el método `evaluarDesafioRespuesta()` rastrea el ciclo de vida completo de cada *Nonce* presente en el AST.

Cuando un agente emite un *nonce* (el desafío), la lógica del motor registra el momento de la creación y además comprueba si, posteriormente dentro del protocolo, el receptor devuelve ese mismo *nonce* modificado o encapsulado bajo alguna clave para demostrar su autenticidad (la respuesta). Si se emite un desafío pero nunca se recibe la respuesta, el motor lo marca como una debilidad estructural, pues la comunicación queda expuesta a posibles interceptaciones.

5.3.5 Detección de Riesgo de Reflexión

El ataque de reflexión o de espejo suele estar presente en canales simétricos mal diseñados. Esto ocurre cuando un atacante devuelve el mismo criptograma o desafío exacto a su emisor original, haciéndole creer que es el interlocutor legítimo.

A través del método `evaluarRiesgoReflexion()` se analiza el flujo de los criptogramas simétricos directos. Al detectar que un agente envía un *nonce* y el receptor de este lo devuelve intacto (misma clave y formato estructural), el motor lanza una alerta, informando de ello. Con el fin de evitar este falso positivo, el sistema aprueba la comunicación solo en el caso de detectar una transformación asimétrica (inclusión de otros elementos para alterar la aridad del criptograma) o el uso de claves distintas.

Tras haber realizado un análisis exhaustivo del comportamiento del motor, la detección de reflexión ha sido recalificada al nivel de *Advertencia* estructural en lugar de ser una vulnerabilidad crítica. Se ha tomado esta decisión pues el motor opera a nivel simbólico y para tratar de simular el problema real de la coincidencia de longitudes en bits para permitir una mejor simulación en la aplicación se ha usado como aproximación heurística la posición del *nonce* y aridad de los componentes del bloque cifrado. Al incluir la palabra "Advertencia" en la descripción del *enum* REFLEXION, el método `registrarVulnerabilidad()` del Protocolo detecta automáticamente que no debe marcar el protocolo como inseguro, permitiendo que la información siga visible para el usuario sin penalizar la evaluación del protocolo.

5.3.6 Evaluación de Claves, Suplantación y Autenticación Mutua

El bloque más complejo del motor es el encargado de la simulación del conocimiento y de comprobar la accesibilidad. Su funcionamiento se basa en los métodos `evaluarClavesYSuplantacion()` y `evaluarAutenticacionMutua()`.

En primer lugar, se analiza si algún agente recibe un mensaje cifrado para el cual nunca obtiene la clave de descifrado necesaria. El motor es lo suficientemente robusto como para poder reconocer cuándo el criptograma es un *ticket* opaco destinado a otro interlocutor y cuándo es un defecto de diseño donde necesita usar una clave que no llega a poseer.

En segundo lugar, si el motor ha detectado que un KDC distribuye una clave simétrica pero, tras finalizar el intercambio de mensajes, una de las partes nunca ha usado dicha clave con el fin de cifrar un paquete y reenviarlo a la otra parte, el sistema reconoce que existe una "Falta de Autenticación

Mutua". Sin embargo, tras realizar un análisis de los protocolos seguros unidireccionales, el motor clasifica este evento como una *Advertencia* estructural en lugar de un fallo crítico, evitando así arrojar falsos positivos de inseguridad en la evaluación global del protocolo para estos casos, mientras sigue alertando al usuario sobre la viabilidad teórica de un ataque de suplantación si la comunicación no está diseñada para ser unidireccional.

Una vez se ha completado el análisis, si el objeto `Protocolo` ha logrado superar todos los filtros sin registrar vulnerabilidades lógicas, el `MotorAnálisis` sentencia que se cumplen las exigencias de seguridad formal y devuelve un veredicto positivo, delegando en las fachadas superiores la persistencia de este resultado en la base de datos.

5.3.7 Resultado Final de la Interfaz

A continuación se presenta el resultado final de la implementación, contrastando el diseño conceptual de los *mockups* con la aplicación funcional construida.

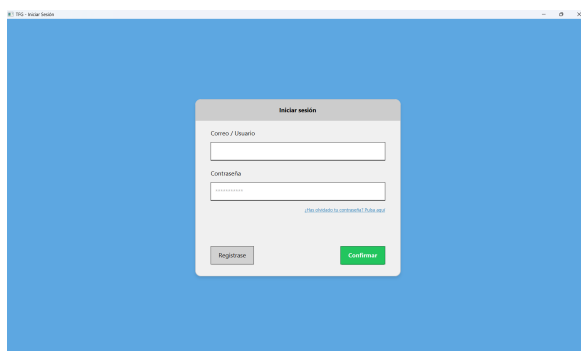


Figura 5.1. Pantalla de inicio de sesión final implementada en JavaFX.

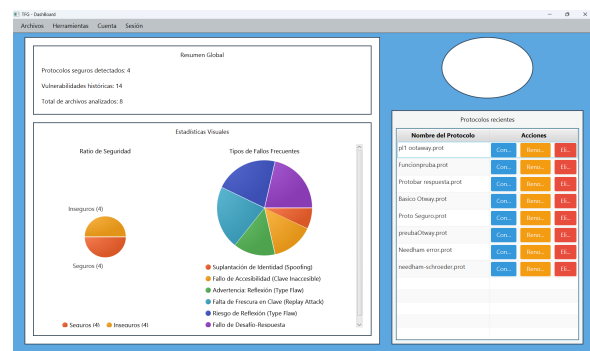


Figura 5.2. Panel de control (Dashboard) con historial de protocolos cargado.

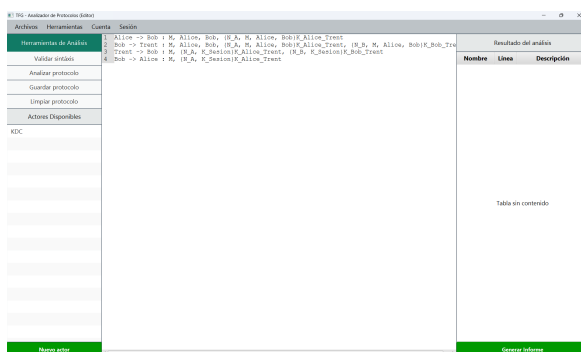


Figura 5.3. Editor de protocolos con especificación cargada y numeración de líneas.

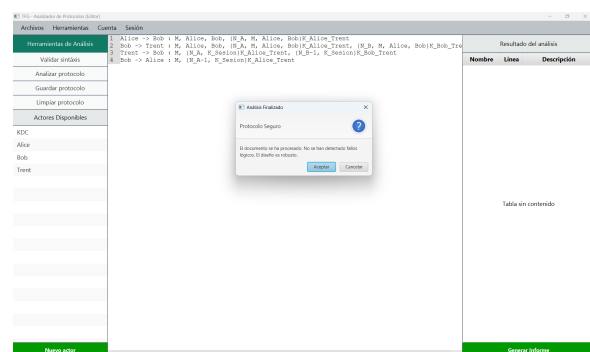


Figura 5.4. Panel de resultados mostrando veredicto positivo tras el análisis.

6

Pruebas y Verificación

En este capítulo se trata el marco metodológico y operativo pensado para evaluar la integridad, fiabilidad y robustez del software presentado. El enfoque usado para la estrategia de verificación se articula sobre la hibridación de la rigurosidad de las pruebas de caja blanca sobre el núcleo algorítmico, con la verificación funcional y empírica de la interfaz gráfica de usuario. El objetivo final es certificar de manera medible el cumplimiento de los requisitos de software establecidos en las fases iniciales del proyecto.

6.1 Trazabilidad de Requisitos

La siguiente tabla relaciona cada requisito funcional con el Caso de Uso que lo implementa y la prueba de verificación que certifica su correcto funcionamiento, garantizando la cobertura completa del sistema.

Requisito	Descripción	Caso de Uso	Verificación
RF1.1	Registrar credenciales	CU01	Manual (SMTP)
RF1.2	Iniciar sesión	CU02	Manual
RF1.3	Cerrar sesión	CU04	Manual
RF1.5	Cambiar contraseña	CU07	Manual
RF1.6	Eliminar cuenta	CU06	Manual
RF1.7	Comprobar contraseña	CU02	<code>testGeneraSaltoBCryptYModeloUsu()</code>
RF1.8	Recuperar contraseña	CU03	Manual (SMTP)
RF2.1–2.3	Dashboard y estadísticas	CU15	Manual
RF2.4–2.7	Historial de protocolos	CU15–CU18	Manual
RF3.1	Editor de texto	CU08	<code>testCompilarProtocoloValido()</code>
RF3.3	Numeración de líneas	CU08	Manual (RichTextFX)
RF7.1	Validar sintaxis	CU10A	<code>testRechazarSintaxisInvalida()</code>
RF7.2	Ejecutar motor de análisis	CU11	<code>testYahalomParalelo()</code>
RF7.3	Identificar suplantación	CU11	<code>testDetectarRiesgoDeReflexion()</code>
RF7.11	Autenticación unilateral	CU11	<code>testAutenticUnilatEsAdv()</code>
RF7.12	Replay Attacks (frescura)	CU11	<code>testDetectarFaltaDeFrescura()</code>
RF6.2	Desafío-Respuesta	CU11	<code>testDesafioRespuestaAjena()</code>
RF5	Identificar texto cifrado	CU11	<code>testParseoCompositeCifradoAnidado()</code>
RF6.1	Gestión de nonces	CU11	<code>testParseoNoncesYClaves()</code>
RF8.1	Exportar protocolo	CU11	Manual
RF8.2	Importar protocolo	CU12	Manual
RF8.3	Autoguardado	CU11	Manual
RF9	Generar informe PDF	CU14	Manual
RF11	Confirmar acciones irreversibles	CU06, CU16	Manual
RNF5	Cifrado de credenciales	CU01, CU07	<code>testGeneracionSaltoBCryptYModeloUsuario()</code>
RNF2	Robustez del analizador	CU10	<code>testRechazarSintaxisInvalida()</code>

Tabla 6.1. Tabla de trazabilidad: Requisito → Caso de Uso → Prueba de verificación.

6.2 Metodología de Pruebas

La metodología de pruebas unitarias adoptada en la fase de verificación del sistema se basa en el patrón de aislamiento unitario e integración continua. El desarrollo del repertorio de pruebas se ha implementado utilizando el *framework* **JUnit 5** (Jupiter API), coordinado bajo el ciclo de vida del gestor de construcción corporativo **Apache Maven** mediante el plugin *Surefire*.

La ejecución de estas pruebas se realiza bajo el entorno de ejecución integrado proporcionado por Java 25 LTS, aislando el comportamiento matemático y sintáctico del sistema a través de dos estrategias fundamentales de ingeniería:

1. **Aislamiento del Entorno de Persistencia:** Los componentes lógicos encargados de evaluar la seguridad de los protocolos actúan sobre estructuras de datos locales instanciadas directamente en memoria. Esto evita realizar escrituras, lecturas o modificaciones concurrentes en el archivo físico binario de *SQLite* (`analisis.db`), garantizando la naturaleza determinista e idempotente de cada prueba (cada experimento arranca y concluye en un estado completamente controlado y limpio).
2. **Desacoplamiento del Hilo Gráfico:** El motor de análisis y el analizador léxico-sintáctico se evalúan de forma independiente a la capa de presentación de *JavaFX*. De este modo, la suite de pruebas unitarias certifica las reglas de inferencia lógica del modelo de *Dolev-Yao* en un entorno multihilo sin necesidad de instanciar el hilo principal de renderizado de la interfaz (*JavaFX Application Thread*), optimizando drásticamente los tiempos de computación y permitiendo su ejecución directa desde la interfaz de comandos (*CLI*) mediante el comando formal:

```
./mvnw.cmd clean test
```

Clase de Prueba	Método	Requisito	Resultado
GestorUsuariosTest	testGeneracionSaltoBCryptYModeloUsuario()	RNF5, RF1.7	PASS
AnalizadorLexicoTest	testCompilarProtocoloValido()	RF3.1, RF4	PASS
	testRechazarSintaxisInvalida()	RF7.1, RNF2	PASS
AnalizadorSintacticoTest	testParseoNoncesYClaves()	RF6.1	PASS
	testParseoCompositeCifradoAnidado()	RF5	PASS
MotorAnalisisTest	testDetectarFaltaDeFrescura()	RF7.12, RF6.1	PASS
	testDetectarRiesgoDeReflexion()	RF7.3	PASS
	testYahalomParalelo()	RF7.4	PASS
	testDesafioRespuestaAjena()	RF6.2	PASS
	testAutenticacionUnilateralEsAdvertencia()	RF7.11	PASS
	testOtwayReesTypeFlaw()	RF7.5	PASS

Tabla 6.2. Inventario completo de la suite de pruebas unitarias y resultado de ejecución con Apache Maven Surefire.

6.3 Casos de Verificación

En esta sección queda documentado el comportamiento real del sistema ante diferentes escenarios de prueba críticos, contrastando el diagnóstico automatizado del *backend* (registrado en los cuadernos de pruebas de JUnit 5) con la respuesta interactiva que la interfaz visual desarrollada con *Scene Builder* expone al usuario final.

6.3.1 Verificación Léxico-Sintáctica y Manejo de Errores de Entrada

El primer nivel de defensa de la herramienta consiste en asegurar la tolerancia a fallos ante especificaciones de protocolos malformadas o corruptas introducidas por el usuario.

Para validar este comportamiento, se diseñó la prueba unitaria `testRechazarSintaxisInvalida()`, la cual inyecta deliberadamente una cadena de texto sin el agente receptor de la comunicación ("Alice -> : {Na}K_ab"). El componente `AnalizadorLexico` captura la anomalía a través de sus expresiones de coincidencia regular y aborta la compilación del protocolo lanzando de manera controlada la excepción nativa `AnalizadorException`.

En la interfaz gráfica de usuario, este mecanismo de excepción es capturado por la capa del controlador de la vista, interrumpiendo el flujo de procesamiento y desplegando un cuadro de diálogo

contextual de advertencia que impide la corrupción del árbol sintáctico en memoria, tal y como se ilustra en la Figura 6.1.

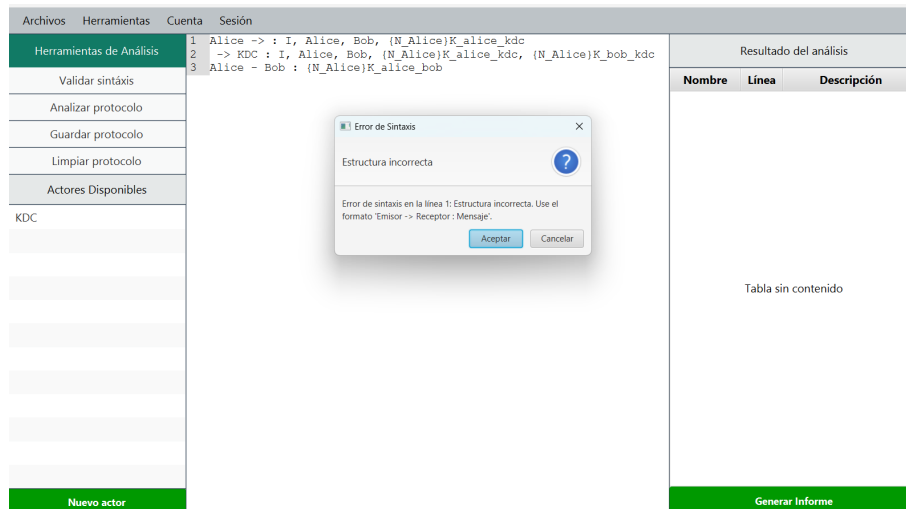


Figura 6.1. Captura de pantalla de la aplicación interceptando y notificando un error de sintaxis en la especificación del protocolo.

6.3.2 Detección de Ataques Reales mediante el Motor Analítico

Para validar la veracidad diagnóstica del **Motor Analisis**, el sistema fue sometido a la evaluación de dos de las vulnerabilidades más críticas en el ámbito de los protocolos criptográficos de comunicación:

6.3.2.1 Escenario A: Ataque de Repetición por Falta de Frescura

Se evaluó un protocolo donde el servidor de autenticación distribuye una clave de sesión secreta compartida entre dos agentes, pero omitiendo la inclusión de marcas de tiempo o números aleatorios únicos (*nonces*) previamente generados por los destinatarios ("KDC -> Alice : {K_alice_bob}K_alice_kdc").

La prueba unitaria `testDetectarFaltaDeFrescura()` verificó de manera automatizada que el motor procesa el mensaje y añade un objeto del tipo `Vulnerabilidad` clasificado como "Advertencia Estructural (Falta de Desafío-Respuesta)". La confirmación visual de esta vulnerabilidad se expone en la interfaz gráfica, donde el semáforo de seguridad conmuta a estado crítico (Inseguro) y se genera de forma dinámica el listado detallado de los fallos estructurales del mensaje (véase la Figura 6.2).

Archivos	Herramientas	Cuenta	Sesión
1 KDC -> Alice : {K_alice_bob}K_alice_kdc			
Herramientas de Análisis			Resultado del análisis
Validar sintaxis			
Analizar protocolo			
Guardar protocolo			
Limpiar protocolo			
Actores Disponibles			
KDC			
Alice			
Nuevo actor			Generar Informe

Figura 6.2. Visualización en la interfaz del sistema tras detectar un fallo por falta de garantías de frescura en el intercambio de claves.

6.3.2.2 Escenario B: Ataque de Espejo o Reflexión

Se configuró un escenario de prueba formal donde un atacante intercepta un desafío criptográfico emitido por un agente legítimo y se lo reenvía de vuelta simulando ser el receptor válido. El objetivo de este ataque es forzar al emisor original a resolver el enigma utilizando la misma clave criptográfica compartida. La especificación introducida en el analizador fue la siguiente:

```
"Alice -> Bob : {N_alice}K_alice_bob"
"Bob -> Alice : {N_alice}K_alice_bob"
```

La suite de pruebas automatizadas certificó el rigor deductivo del motor de análisis bajo las directrices del modelo de Dolev-Yao. El sistema identificó de forma precisa que el agente remitente del segundo mensaje ejecutaba una acción de transmisión utilizando una estructura de cifrado inaccesible para sus privilegios contextuales actuales. Como resultado, el motor interrumpió la verificación y registró de inmediato la vulnerabilidad de "Suplantación de Identidad (Spoofing)". La representación en el panel principal del software de este ataque masivo por reflexión se detalla visualmente en la Figura 6.3.

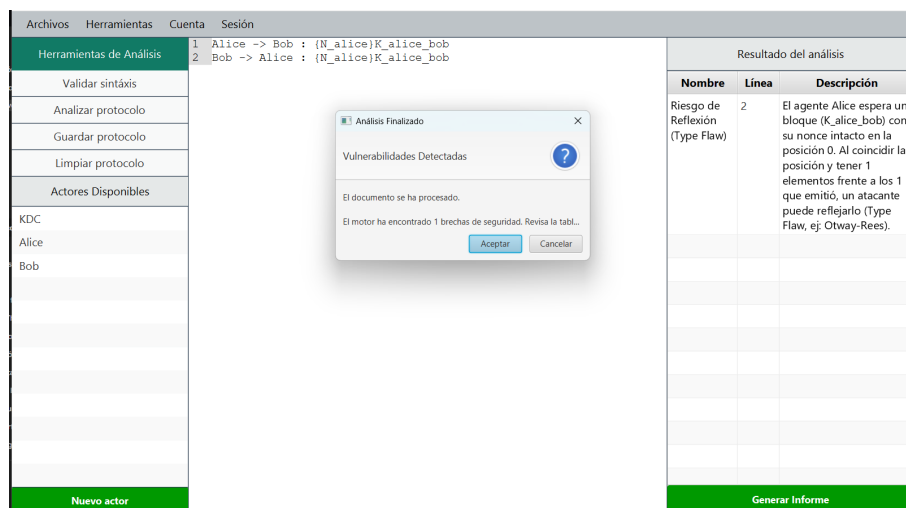


Figura 6.3. Diagnóstico interactivo de la GUI detallando la mitigación y alerta del ataque de suplantación de identidad por reflexión.

6.3.3 Seguridad Local y Persistencia de Credenciales

Para garantizar el estricto cumplimiento de los requisitos de confidencialidad y protección de datos locales, se diseñó un escenario de verificación unitaria sobre la capa del **GestorUsuarios**. El objetivo fue auditar empíricamente el comportamiento del algoritmo adaptativo *jBCrypt* [8] durante el proceso de persistencia.

La prueba unitaria verificó con éxito que el sistema aplica un *salt* criptográfico de forma aleatoria y correcta, generando *hashes* únicos e irrepetibles para ser almacenados en la base de datos *SQLite*, incluso al procesar contraseñas idénticas en texto plano. Esta verificación certifica que el modelo de datos nunca expone credenciales sensibles en memoria y previene vulnerabilidades de almacenamiento estructural, tales como los ataques basados en tablas precomputadas (*Rainbow Tables*).

6.3.4 Verificación Algorítmica Continua mediante TDD

Además de las pruebas de usabilidad y aislamiento, se implementó una extensa batería de pruebas unitarias basadas en *Test-Driven Development* (TDD) para validar la robustez de la lógica heurística del motor. Alimentando el Árbol Sintáctico Abstracto a través del analizador léxico, se verificó la resolución matemática de protocolos complejos en fracciones de segundo y sin intervención de la interfaz gráfica.

Podemos destacar la verificación del protocolo *Yahalom* [7] (verificando la comprensión de la

frescura en bloques cifrados paralelos), la detección de fallos estructurales o *Type Flaws* [7] en el protocolo *Otway-Rees* [7], y la evaluación de suplantación al responder a un desafío con una clave ajena a la comunicación. Esta batería garantiza la invulnerabilidad del núcleo del *software* frente a regresiones en futuras etapas de desarrollo, avisando de cualquier problema.

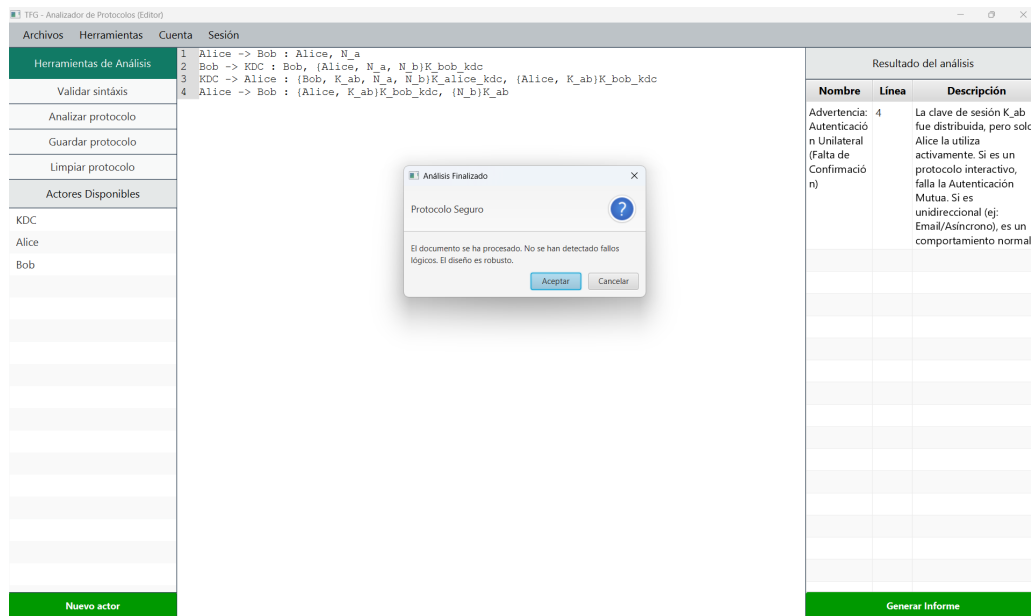


Figura 6.4. Interfaz de la aplicación mostrando el veredicto Seguro para el protocolo Yahalom tras superar todos los filtros del motor analítico.

7

Conclusiones y Trabajo Futuro

7.1 Conclusiones

El desarrollo de este Trabajo de Fin de Grado ha resultado en la construcción exitosa de una herramienta para el análisis de seguridad en protocolos de distribución de claves simétricas, cumpliendo exactamente con los objetivos generales y específicos planteados al inicio del proyecto.

En primer lugar, se ha logrado democratizar el acceso a la verificación formal. Al sustituir los complejos lenguajes de especificación tradicionales por una notación estándar (*Alice y Bob*) más accesible y proporcionar al sistema de una interfaz gráfica intuitiva desarrollada en JavaFX, además se redujo drásticamente la curva de aprendizaje, obteniendo así una herramienta altamente viable para entornos académicos y de auditoría rápida.

Desde el punto de vista arquitectónico, la implementación estricta del patrón Modelo-Vista-Controlador (MVC) [9] ha garantizado un sistema altamente desacoplado. El motor de análisis interno ha demostrado ser computacionalmente robusto: el analizador léxico previene inyecciones sintácticas, mientras que el analizador de descenso recursivo es capaz de modelar el Árbol Sintáctico Abstracto (AST) empleando el patrón *Composite*, resolviendo de manera elegante el problema del anidamiento criptográfico múltiple.

A nivel de seguridad lógica, los algoritmos implementados bajo las premisas del modelo del intruso de Dolev-Yao han probado su eficacia empírica mediante pruebas unitarias (*Caja Blanca*), detectando de manera precisa vulnerabilidades críticas como ataques de repetición (*Replay Attacks*), suplantación

de identidad (*Spoofing*) y falta de garantías de frescura en las claves de sesión. Todo ello apoyado por un ecosistema de persistencia local seguro en SQLite, que salvaguarda la confidencialidad de los usuarios mediante el algoritmo de derivación de claves *jBCrypt*.

En definitiva, este proyecto no solo constituye una solución funcional a un problema real en el ámbito de la ciberseguridad, sino que representa la consolidación práctica de las competencias de ingeniería de *software*, criptografía y procesadores del lenguaje adquiridas durante el Grado en Ingeniería Informática.

7.2 Trabajo Futuro

Debido a la naturaleza modular y escalable del sistema, el proyecto sienta una base sólida para futuras expansiones. Se plantean las siguientes líneas de mejora técnica a futuro:

- **Soporte para criptografía asimétrica:** Extender el analizador léxico-sintáctico y las reglas deductivas del motor para soportar algoritmos de infraestructura de clave pública (PKI), permitiendo evaluar protocolos basados en firmas digitales y certificados.
- **Resaltado de sintaxis interactivo:** Optimización de la interfaz de usuario mediante la implementación de un motor de resaltado de sintaxis asíncrono en tiempo real para el componente `CodeArea`. Esta funcionalidad quedó postergada en el alcance inicial del proyecto para priorizar la estabilidad del hilo principal de JavaFX frente a la carga computacional de las expresiones regulares recurrentes, proyectándose su integración mediante un esquema de hilos de ejecución de baja prioridad.
- **Exportación a lenguajes formales estandarizados:** Añadir un módulo de traducción capaz de convertir el Árbol Sintáctico Abstracto a lenguajes como HLPSL o *spdl*. De este modo, la herramienta podría actuar como una interfaz amigable (GUI) generadora de *scripts* para simuladores de redes más pesados como AVISPA o Scyther.
- **Mitigación de Falsos Negativos por Desalineamiento Binario (Trampa de los Bytes):** Actualmente, el motor de análisis emplea una poderosa heurística de reconocimiento estructural para detectar ataques de reflexión o *Type Flaws*. Sin embargo, tiene el problema de no llegar a analizar el protocolo a nivel binario en memoria RAM. Como trabajo futuro, se plantea incorporar un módulo de verificación de longitudes de campo que mitigue la "Trampa de los Bytes",

previniendo escenarios teóricos donde un atacante pudiera aprovechar que distintas variables consecutivas coincidan casualmente con la longitud en *bytes* del bloque cifrado que se espera.

- **Refactorización del motor mediante el patrón *Strategy*:** Actualmente, las reglas de evaluación del motor están implementadas como métodos internos de la clase `MotorAnalysis`. Como mejora futura, se propone encapsular cada regla en una clase propia que implemente una interfaz común (`ReglaAnalysis`), permitiendo añadir o desactivar reglas sin modificar el código existente.
- **Configuración dinámica de entidades de confianza:** Actualmente, el motor reconoce como servidores de confianza únicamente a "KDC" y "Trent" mediante un conjunto estático. Se plantea permitir al usuario definir entidades de confianza personalizadas a través de la interfaz, ampliando la versatilidad del análisis a protocolos con nomenclaturas no convencionales.

A

Diagramas de Secuencia Adicionales

En este anexo se recogen los diagramas de secuencia correspondientes a los casos de uso secundarios y de soporte del sistema. Estos flujos de ejecución complementan al proceso principal de análisis de protocolos detallado en el cuerpo principal del trabajo.

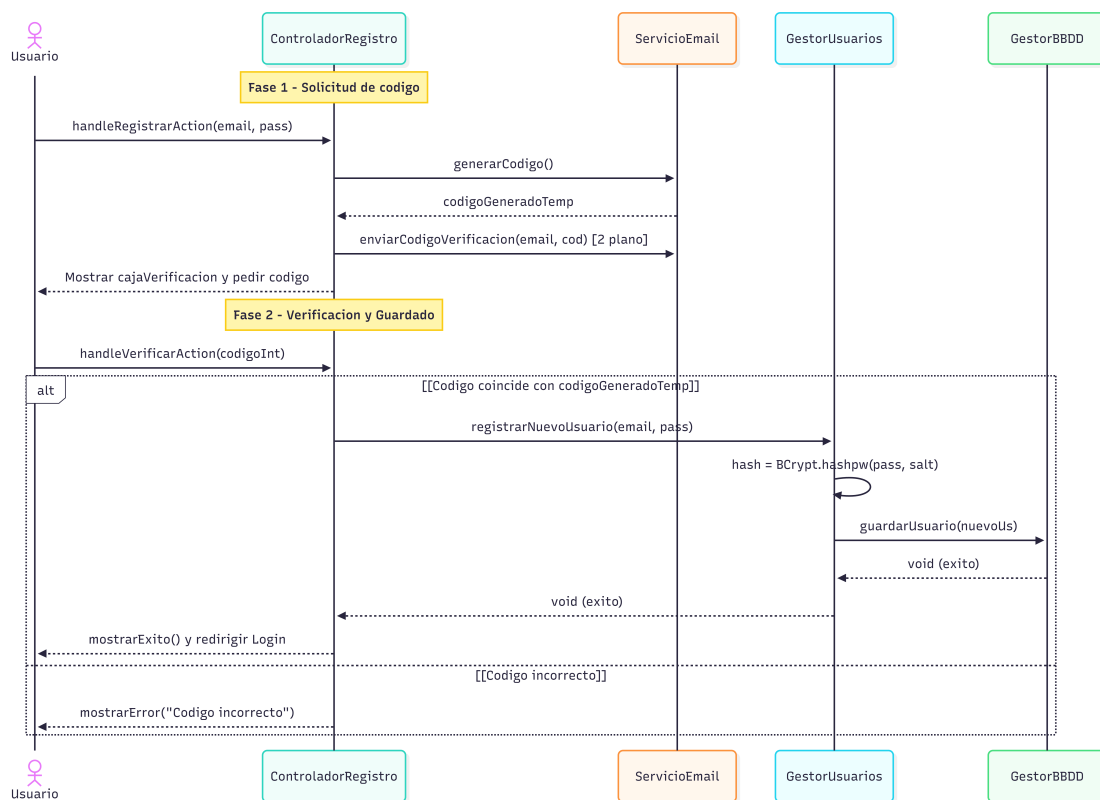


Figura A.1. Diagrama de secuencia correspondiente al proceso de Registro de Usuario CU01.

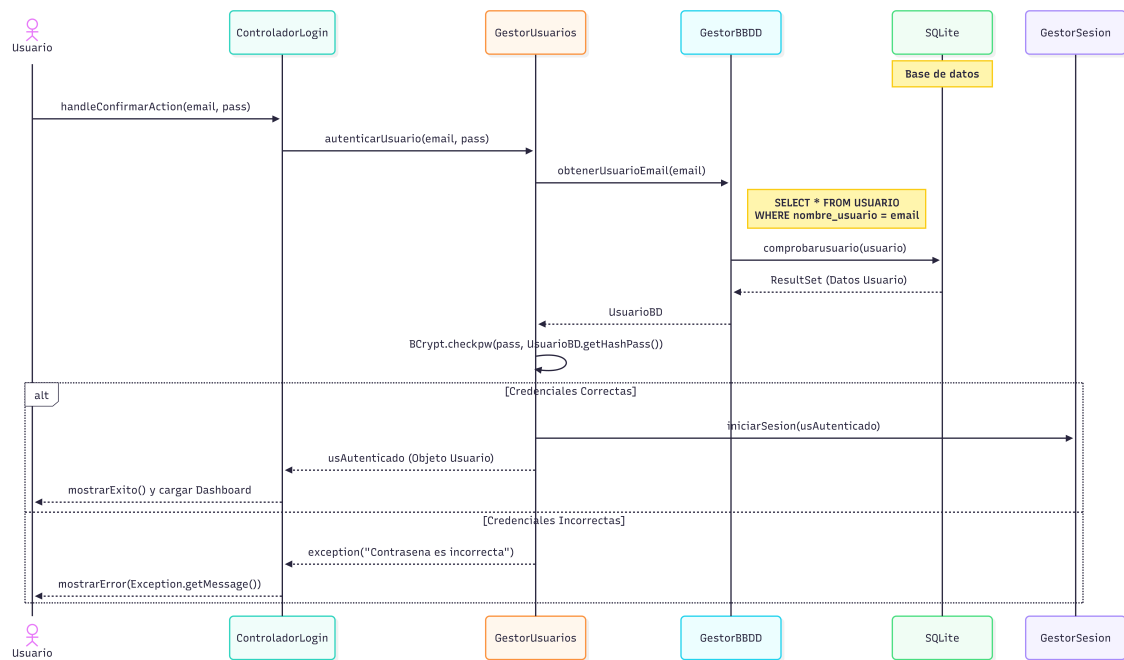


Figura A.2. Diagrama de secuencia correspondiente al proceso de Inicio de Sesión CU02.

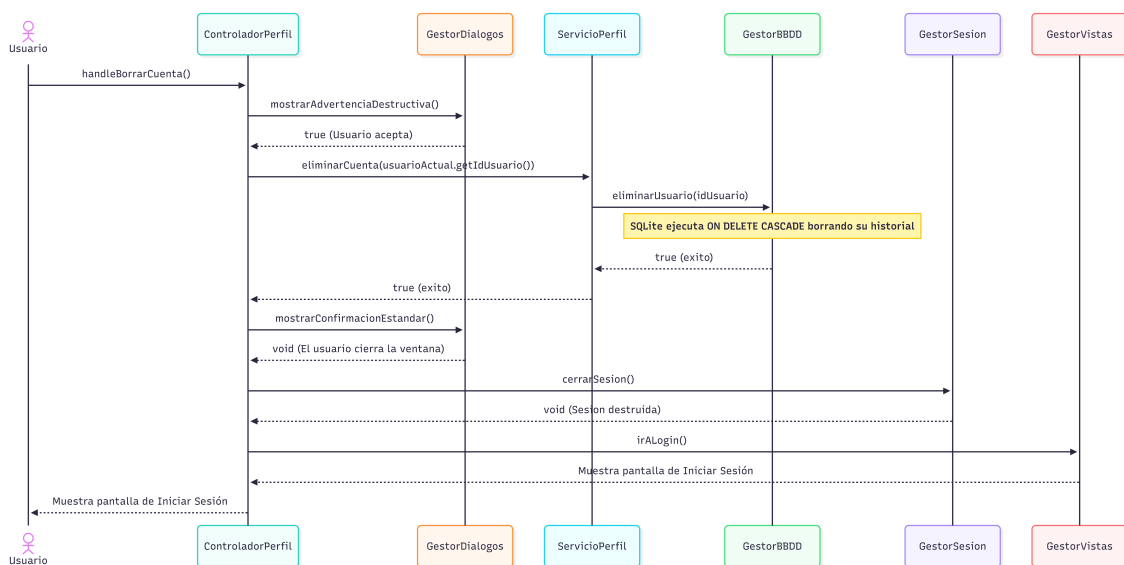


Figura A.3. Diagrama de secuencia correspondiente al proceso de Eliminación de Cuenta CU06.

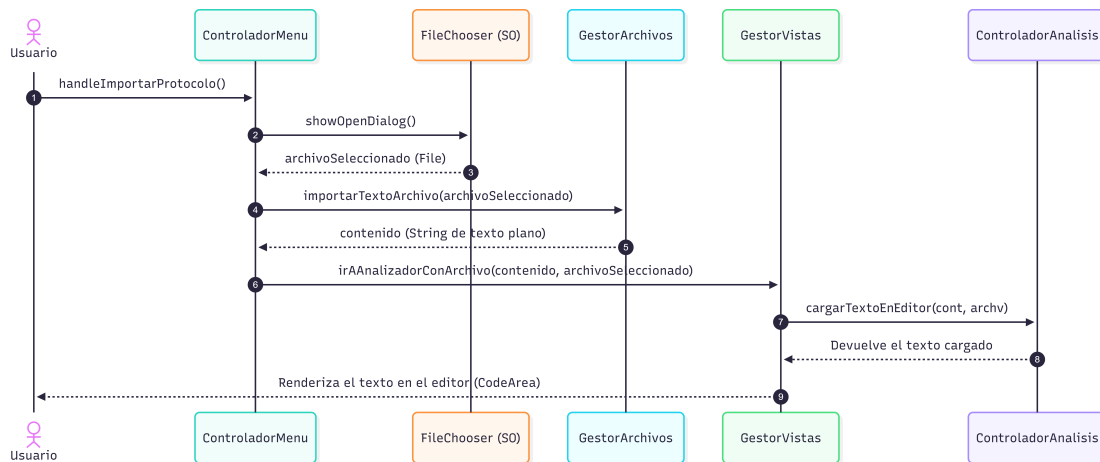


Figura A.4. Diagrama de secuencia correspondiente al proceso de Importación de un Protocolo CU12.

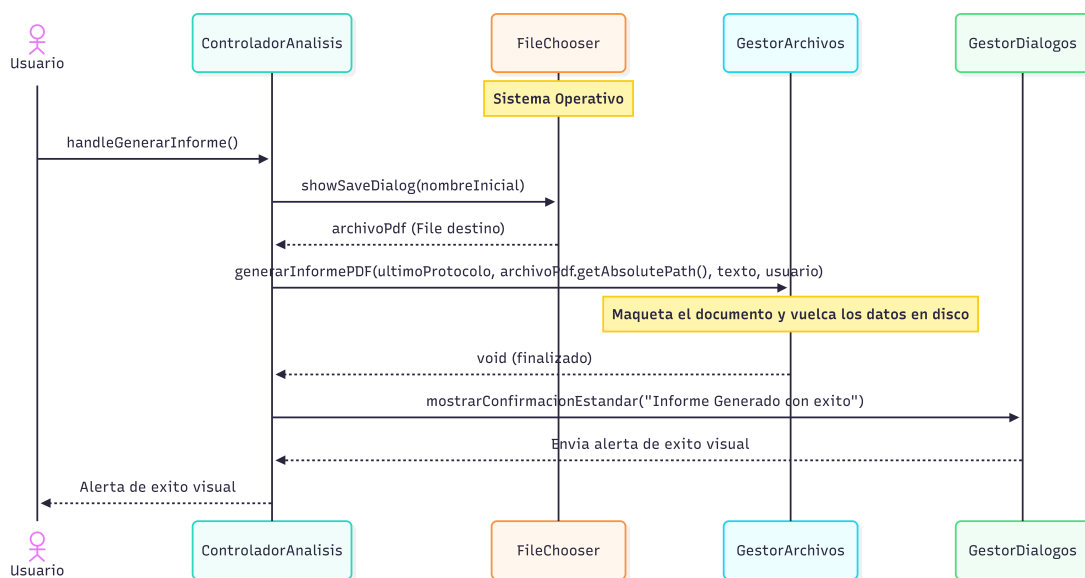


Figura A.5. Diagrama de secuencia correspondiente al proceso de Generación de Informes en PDF CU14.

B

Diagrama de clases de Implementación

Este anexo contiene el diagrama de clases completo de la arquitectura del proyecto y cada uno de sus paquetes a detalle. Debido a la alta densidad de componentes y a la división por capas del motor, el diagrama se presenta en formato apaisado en la página siguiente para garantizar su correcta legibilidad.

89

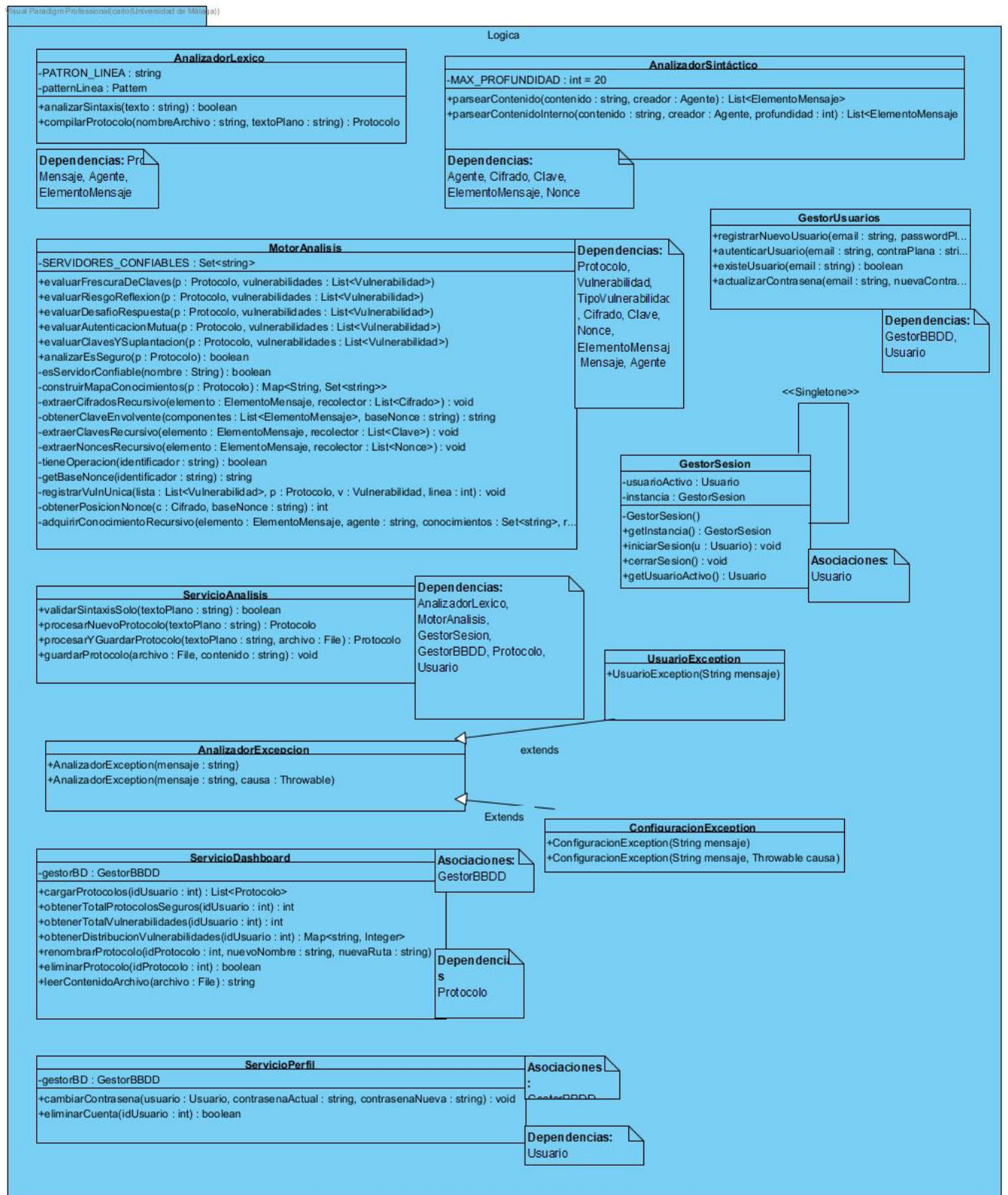


Figura B.3. Diagrama de clases de implementación y arquitectura por capas paquete lógica.

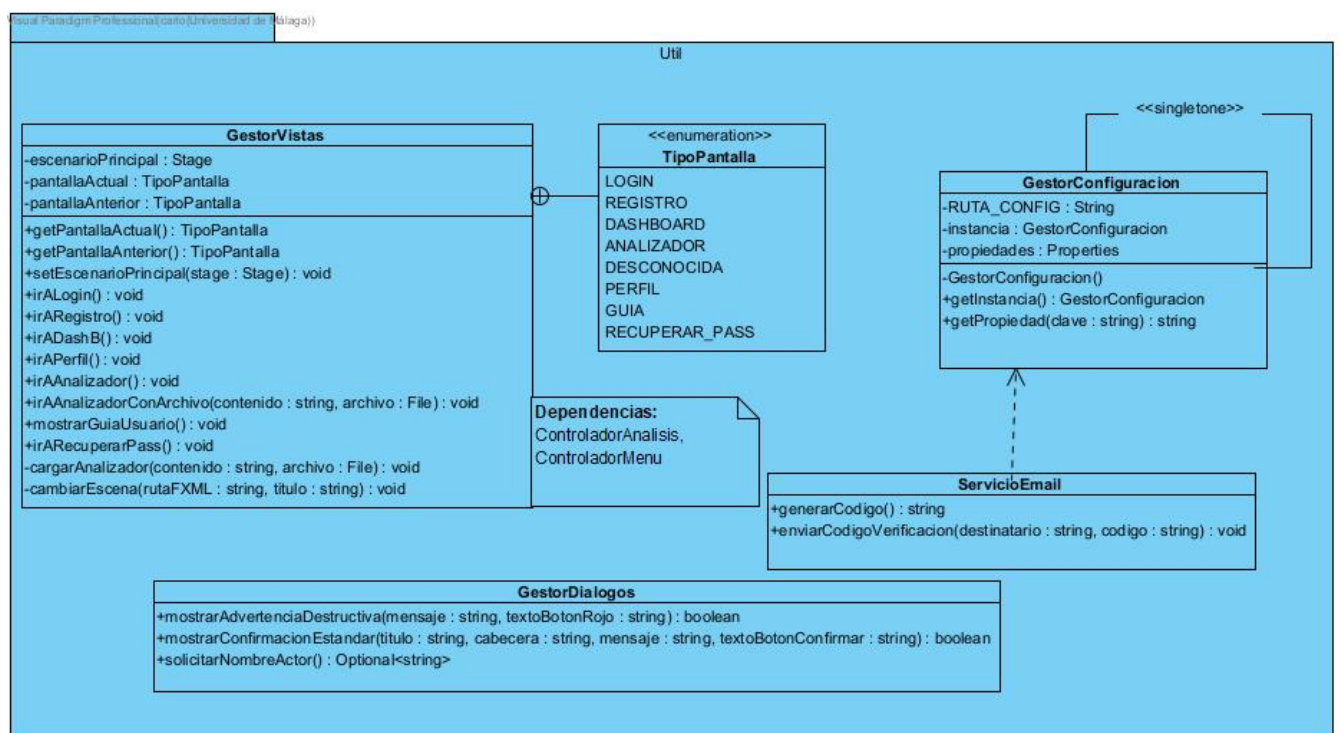


Figura B.4. Diagrama de clases de implementación y arquitectura por capas paquete útil.

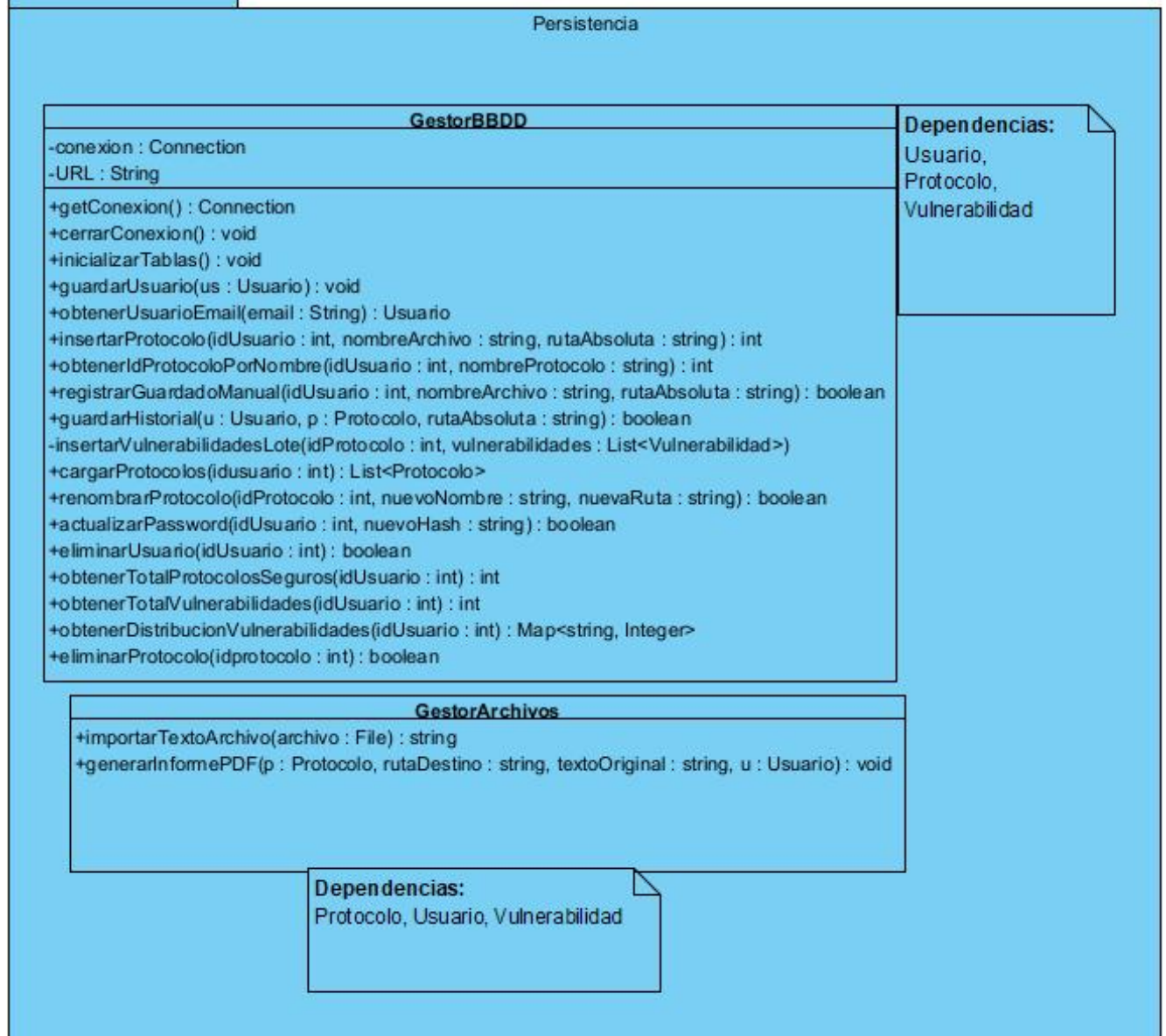


Figura B.5. Diagrama de clases de implementación y arquitectura por capas paquete persistencia.

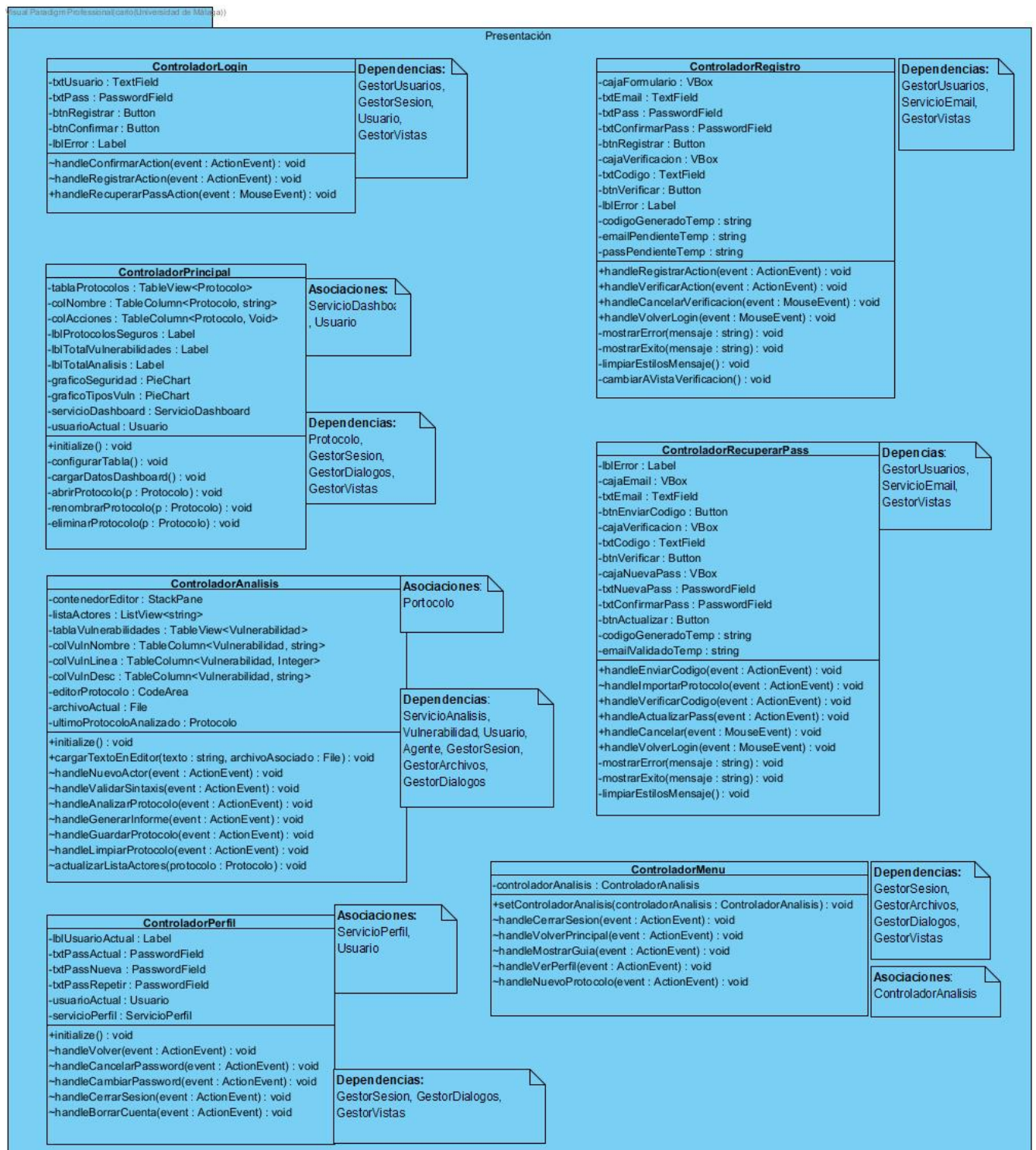


Figura B.6. Diagrama de clases de implementación y arquitectura por capas paquete presentación.



Diagrama del AST

En este anexo se expone el diagrama de clases detallado correspondiente al Analizador Sintáctico y a la estructura del Árbol Sintáctico Abstracto (AST). En él se observa la implementación del patrón de diseño *Composite*, el cual resulta fundamental para resolver computacionalmente la recursividad y el anidamiento múltiple de los mensajes criptográficos evaluados por el sistema.

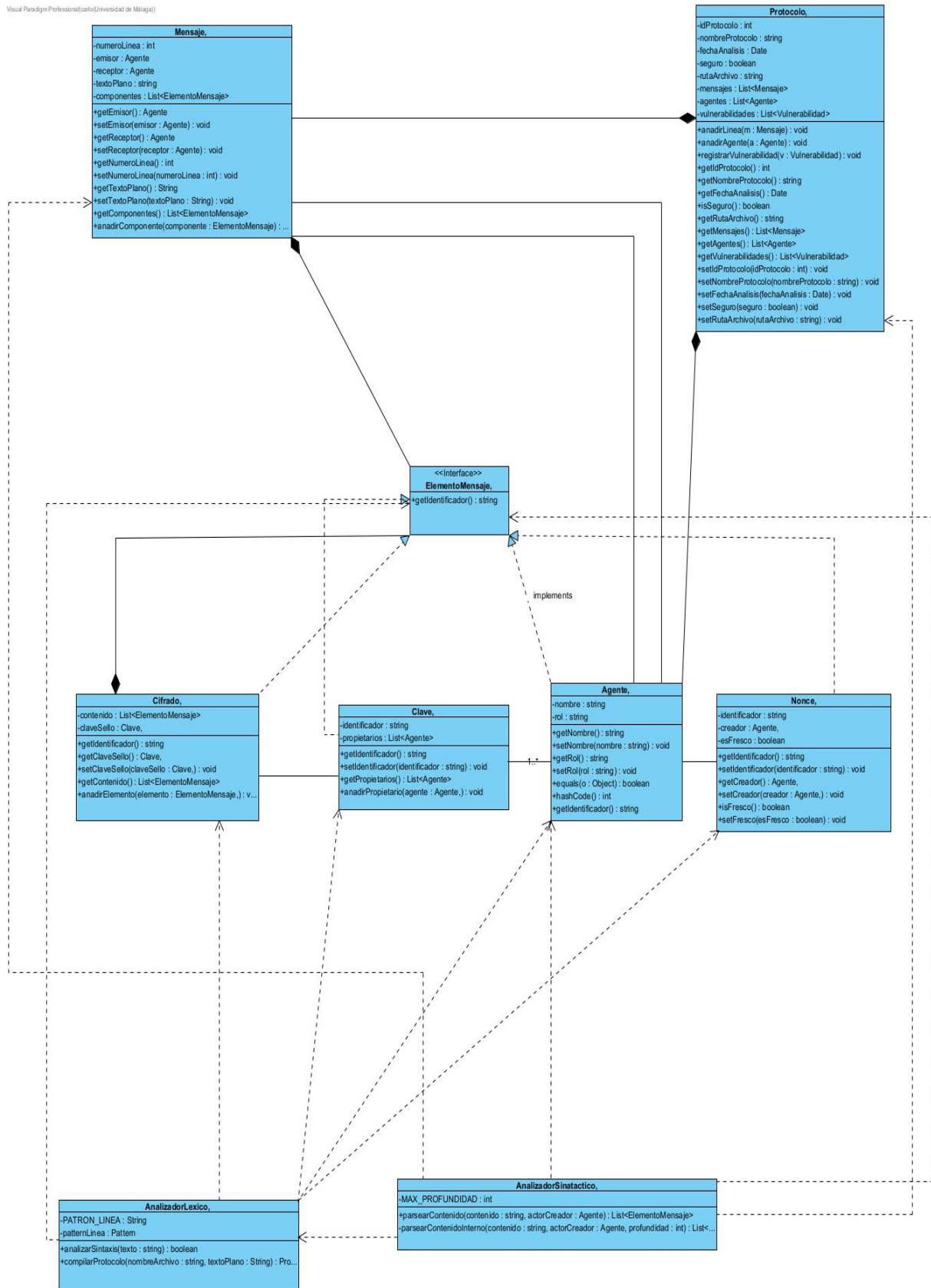


Figura C.1. Diagrama de clases del Analizador Sintáctico y la estructura del AST (Patrón Composite).

D

Registro del Trello

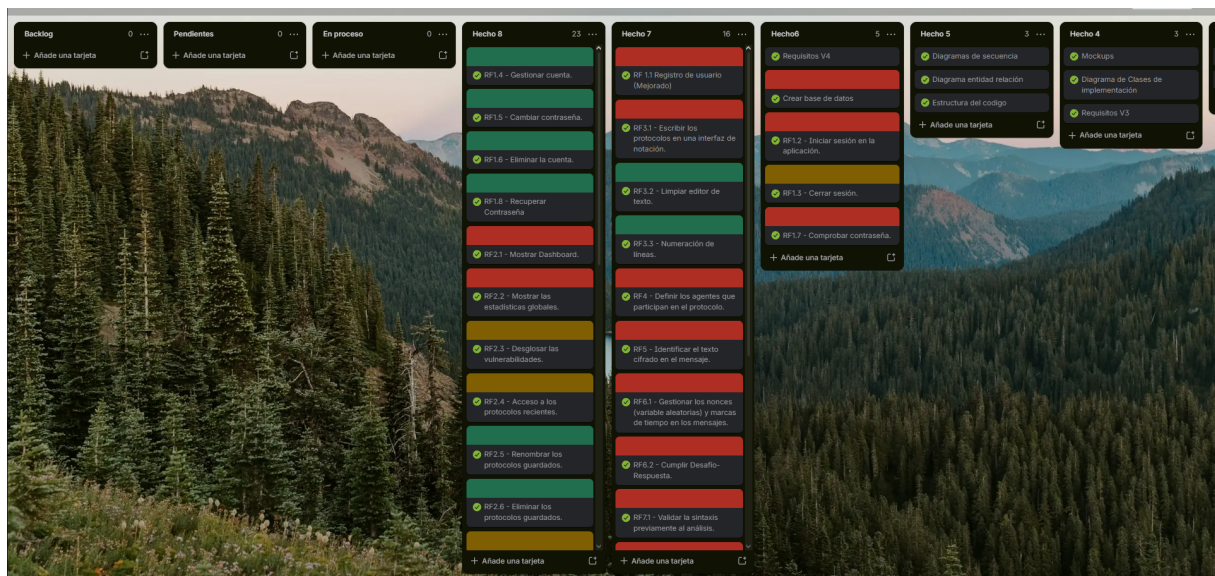


Figura D.1. Registro de los sprints llevados a cabo durante la elaboración del proyecto.

Bibliografía

- [1] A. Armando et al. "The AVISPA Tool for the Automated Validation of Internet Security Protocols and Applications". In: *Computer Aided Verification*. Ed. by Kousha Etessami and Sriram K. Rajamani. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 281–285.
- [2] D. Dolev and A. Yao. "On the security of public key protocols". In: *IEEE Transactions on Information Theory* 29.2 (1983), pp. 198–208.
- [3] Ralph Johnson Erich Gamma Richard Helm and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. URL: <https://www.javier8a.com/itc/bd1/articulo.pdf>.
- [4] D. Richard Hipp. *SQLite Official Documentation*. SQLite Consortium. 2024. URL: <https://www.sqlite.org/docs.html>.
- [5] *JavaFX 21 Documentation*. Oracle Corporation. 2023. URL: <https://openjfx.io/>.
- [6] Gavin Lowe. "Breaking and fixing the Needham-Schroeder Public-Key Protocol using FDR". In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Tiziana Margaria and Bernhard Steffen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 147–166.
- [7] Universidad de Málaga. *TEMA 3 - PARTE 1 ESQUEMAS, PROTOCOLOS Y MECANISMOS DE SOPORTE (A LA SEGURIDAD DE APLICACIONES Y DE REDES)*. Universidad de Málaga.
- [8] David Mazières Niels Provos. "A Future-Adaptable Password Scheme". In: *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. Monterey, California, USA: USENIX Association, 1999. URL: <https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf>.
- [9] Trygve Reenskaug. "Models-Views-Controllers". In: *Technical Note, Xerox PARC* (1979). URL: <https://folk.universitetetioslo.no/trygver/1979/mvc-2/1979-12-MVC.pdf>.